Final Project for SW Engineering CSC648-848 Spring 2023

Team 04

StudentSources

Saru Shrestha (sshrestha11@sfsu.edu)
Kilian Kistenbroker
Elias Abay
Parth Panchal
Joshua Del Rosario
Wilson Yin
Omar Thiongane

URL: http://54.215.216.18/

May 25, 2023

Table Of Contents

Product Summary....4
Milestone Documents M1 - M4....8
Milestone 1....9
Executive Summary....10
Main Use Cases....12
Main Use Cases Diagram....17
Main Data Items & Entities....19
Functional Requirements....20
Non-Functional Requirements....26
Competitive Analysis....30
Important features of competitive products....30
Key Features of Competitors vs. StudentSources....31
Competitive Analysis Summary....31
High-level System Architecture and Technologies Used....32
Milestone 2....33
Data Definitions....34
Prioritized Functional Requirements....36
UI Mockups and Storyboards....42
High level database architecture and organization....55
Database Organization....55
Media Storage....60
High Level APIs and Main Algorithms....62
High Level UML Diagrams....63
High Level Application Network and Development Diagrams....64
Key Risks in Current Work....65
Project Management....67
Milestone 3....68
Data Definitions V3....69
Functional Requirements V3....71
Wireframes V2....77
To Upload and Open A File....77
To Message Other User....79
To Search Other User's resources....80
High Level Database Architecture and Organization V2....81
Database Organization....81
Entity Relationship Diagram (ERD)....86
Enhanced Entity Relationship (EER) Diagram....87
Database Management System (DBMS)....88

Media Storage....88
Search/filter architecture and implementation....88
High Level Diagrams V2....89
UML Diagram....89
Deployment and Network Diagram....90
Milestone 4....91
Product Summary....92
Usability test plan....96
Test Objectives Per Function....96
Test Description....97
Usability Test Table: Effectiveness....99
Usability Test Table: Efficiency....102
User Satisfaction....103
QA test plan....108
Test Objectives....108
HW and SW Setup....108
Risks and Contingencies....109
Feature to Be Tested....109
QA Test plan....109
Code Review....111
Coding Style....111
Code Featuring QA and Usability Test....111
Self-check on best practices for security....118
Protected Major Assets....118
Confirmation of Encrypted Password....118
Confirmation of Input Data Validation....119
Self-check: Adherence to original Non-functional specs....122
Team Member Contribution....129
Milestone 1 - Contribution Detail....130
Milestone 2 - Contribution Detail....132
Milestone 3 - Contribution Detail....133
Milestone 4 - Contribution Detail....135
Team Member Contribution Score....138
Team Members' Signature In Agreement of Contribution Score....139
Post Analysis - Lesson Learned....140
Main Challenges....140
Addressing Those Challenges....140

Product Summary

Project Name: StudentSources

Committed Functions

1. A general user shall be able to create one or more accounts.
2. A general user shall be able to sign up with a unique email address.
3. A general user shall be able to sign up with a non-unique password.
4. A general user shall be able to see if all requirements for creating a password are met.
5. A general user shall be able to sign up with a non-unique username.
6. A general user shall be able to read the terms and conditions.
7. A general user shall be able to agree to the terms and conditions.
8. A registered user shall have at least one account.
9. A registered user shall be able to view their account information in the Account page.
10. A registered user shall be able to temporarily unhide account password information in the *Account* page. By default, when a registered user visits the *Account* page, the password would be hidden for security and password.
11. A registered user shall be able to change their account username.
12. A registered user shall be able to change their account email.
13. A registered user shall be able to change their account password.
14. A registered user shall be able to upload any supported file type to their account's cloud storage.
15. A registered user shall be able to navigate through files with a directory tree.
16. A registered user shall be able to organize files alphabetically through a directory tree.
17. A registered user shall be able to view the current directory from a *Files* panel.
18. A registered user shall be able to create directories.
19. A registered user shall be able to create text files.
20. A registered user shall be able to create bookmarks of web URLs.
21. A registered user shall be able to pin the resources for easy access in the future.
22. A registered user shall be able to view supported files on the website.
23. A registered user can view the contents of their files on a *Display* panel.

24. A registered user can view the contents of their saved web pages on a *Display* panel.
25. A registered user shall have a place to type and save a note for each resource.
26. A registered user shall be able to comment on files.
27. A registered user shall be able to delete/move resources to a trash bin.
28. A registered user shall be able to delete resources from a trash bin permanently.
29. A registered user shall be able to recover items from a trash bin.
30. A registered user shall be able to download files onto their computer.
31. A registered user shall be able to configure permissions of their shared directories/folders. The permission allows other users to either only view the list or download the list.
32. A registered user shall be able to share their directories by configuring the list as ‘public’.
33. A registered user shall be able to view other directories shared publicly.
34. A registered user shall be able to download other shared directories onto their computer.
35. A registered user shall be able to search public directories.
36. A registered user shall be able to add other registered users as friends.
37. A registered user shall be able to remove other registered users from friend-list.
38. A registered user shall be able to mark other registered users as their favorite friend.
39. A registered user shall be able to configure visibility of their resources at any time. The visibility can be either public or private.
40. A registered user shall be able to message another registered user.
41. A registered user shall be able to create a group chat to message multiple other registered users.
42. Account shall have a certain amount of cloud storage.
43. Accounts shall require and store a username that is at least 6 characters.
44. Accounts shall require and store a real name tied to them, and the real name must have a first and last name.
45. Accounts shall require and store a password tied to them.
46. Accounts password shall be encrypted when stored in a database.
47. Accounts shall be created by one or more general users.
48. An account shall be owned by one registered user.
49. Resources shall be uploaded by a registered user.
50. Resources shall be organized in an alphabetical order.

51. Resources such as text files shall be able to be created by a registered user.
52. Resources shall be viewed by a registered user.
53. Resources shall be commented by registered users.
54. Resources shall be moved by a registered user to a trash bin.
55. Resources shall be able to be deleted permanently by a registered user from a trash bin.
56. Resources shall be recovered by a registered user from a trash bin.
57. Resources shall be downloaded on the computer by a registered user.
58. Resources shall have comments from other registered users.
59. Resources shall be pinned by a registered user.
60. A resource shall have a note created by a registered user.
61. Resources shall have configured visibility by a registered user.
62. Directory trees shall be navigated by a registered user.
63. Directory trees shall have files organized in alphabetical order.
64. Directories shall be created by a registered user.
65. Directories shall have configured permissions added by a registered user.
66. Shared directories shall be able to be viewed by a registered user.
67. Directories shall be able to be downloaded by other registered users onto their own computer.
68. Directories of other registered users shall be able to be searched by registered users.
69. Comments shall be added on resources by registered users.
70. A friend shall be added by registered users.
71. A friend shall be removed by registered users.
72. A friend shall be added as a favorite by registered users.
73. A note shall be typed and saved by a registered user for each resource.
74. A registered user shall be able to search other registered users whose profile is set public.
75. A registered user shall be able to search friends.
76. A registered user shall be able to search pinned resources.
77. A registered user shall be able to clear notes added to the resource.
78. A registered user shall be able to clear comments added to the resource.
79. A pinned resource shall be searched by registered users.
80. Comments shall be able to be cleared by a registered user at any time.
81. A friend shall be searched by registered users.
82. Notes shall be cleared by registered users at any time.

Unique Features

We have many unique features that make our website, StudentSources, stand out from the rest. The unique features are as follows: a built-in AI chatbot, a group chat feature, and other ways for students to collaborate such as sharable resources and direct messaging. StudentSources' built-in chatbot is a hot new feature that many websites are adding to enhance the user experience and save the user lots of time when searching for an answer. In addition to the AI chatbot, StudentSources promotes collaboration among students. Through the platform, students can easily share resources, exchange ideas, and collaborate on projects. They can comment on each other's work, direct messages to each other, create a group chat as study group, and have the ability to add or remove friends to stay in touch with users with similar classes and resources. These features make our website an ideal tool for students to enhance their academic experiences and store their study resources.

Product URL: http://54.215.216.18/

Milestone Documents M1 - M4

Milestone 1

Executive Summary

When studying for finals, the usual routine is to start reviewing from the first lecture and then go from there. Sometimes it's hard to do so because the resources you used throughout the semester aren't organized well, then you are going back and forth between topics and weeks of lectures. Students will waste countless hours studying just for it to end up not being that effective. We thought a good idea was a website to organize your bookmarks and resources so that it is easier to go back to those resources. Our website, StudentSources, is a website where any resources from students can be uploaded in an organized fashion. The ordered list can then be shared with other students on the website. The list folders could be titled by class name, class number, or professor. This way students can easily find resources to study within just a few clicks, and the time they spend studying will be effective and useful.

StudentSources will have a range of features to make studying easier for students. The main key feature of the website will be being able to organize online resources such as links to any useful resources or videos. Users will be able to put resources in any order they want, resulting in a list of related resources. The website will also accept multiple file formats, such as PDFs, PNGs, and links to MP4s, documents, pictures, videos. Another key feature of StudentSources is the ability to share those lists of resources with other students. This will prove to be useful when studying alone, with a group, or when wanting to share your study strategies with someone else. The last key feature StudentSources has to offer is the ability to add notes onto PDFs to add any additional information you want to save.

StudentSources is an educational platform offering various unique features to help students enhance their knowledge. Sometimes, the resources that students find online might end up using a different method or formula than what the professor teaches. This makes the resource found practically useless, and makes studying feel pointless without the correct study materials. One key feature of StudentSources is its flexibility. The website's structure allows for the organization of resources based on the professor's name, course name, or course abbreviation and number (CSC648 for example). This feature provides an easy and fast way to navigate the website and find what you are looking for. Other features include collaboration tools such as sharing, commenting, direct messaging, and group chats, as well as the ability to add or remove friends. StudentSources encourages collaboration and sharing of knowledge, hence students can search for publicly shared directories to reference another student's notes and resources. They can also leave comments on publicly shared resources, boosting the spread of information. Students can add other users as their friend and even mark them as *Favorite* for easy search. The platform also allows users to direct messages to one

another. Students can also create a group chat for group study as group study is also one of the effective means to ace classes. Lastly, our team is planning to include a chatbot powered by AI technology that can summarize documents, saving students valuable time, instead of reading a document with 100 pages of notes. AI chatbot can assist students with their queries and provide personalized support.

Overall, StudentSources offers an innovative platform for students to access and organize their course materials, and even find new course materials from other students. In addition, StudentSources offers a comprehensive set of features designed to facilitate learning and collaboration among students. Apart from those features, the website stands out to be a sustainable and environment-friendly space as the website supports paperless notes organization. We highlight the convenience, collaboration features, and environmental benefits in our platform. By engaging with student communities, promoting sustainability, and offering personalized and collaborative features, we aim to establish our platform as the top choice for students to organize their study materials while contributing to a greener future.

Main Use Cases

1. Struggling with grades (Aaron, 19, 2nd year in college)

Aaron is from Utah but attends SFSU and lives in the dorms. He wanted to go to SFSU to experience living in SF and to meet new people. He is an avid procrastinator and struggles to study for exams and midterms. Recently, he has been struggling grade-wise, so much so that his parents have threatened to send him back to go to college back home in Utah if he doesn't improve his grades. His main problem is that when finals come around, he keeps playing video games and then studies at the last minute. He also needs help staying motivated because he gets frustrated quickly and gives up on studying after a while. He saw someone sitting beside him studying and asked them what website they were using. The student said the website was called StudentSources, and that they made a list of study resources on it. After using StudentSources, Aaron has resources posted by students in SFSU for his Computer Science and History classes. Because of the ordered list of resources, he can easily stay on track and focused. Aaron was also surprised to see that there is a chatbot he could use, especially since everyone has been talking about AI. The chatbot makes his study time more efficient and less stressful. He gets good grades on his finals and finishes the semester with A's and B's, which impresses his parents. His parents let him stay at SFSU, and he is happy he found StudentSources and gets to stay in San Francisco.

2. About to graduate (Sydney, 23, 4th-year college)

Sydney is a senior student from Tennessee who attends SFSU and is excited to graduate this spring. She always wanted to experience living in a big city, and SFSU has provided her with that opportunity. However, Sydney has been feeling overwhelmed lately, as she is away from home and has many final projects and assignments due at the end of the semester. She has always been a procrastinator, making completing her work on time challenging. Sydney is worried that she may not graduate with the high GPA she had hoped for. She mentioned her struggles to a friend, who recommended using a website called "StudentSources," which is a website where students can upload resources like slides and notes to share with other students. After using StudentSources, Sydney found valuable resources from fellow students for her classes to help her stay on track with her assignments. She has found that she can complete her work on time, stay motivated, and get better grades. With the help of StudentSources, Sydney is confident that she will graduate with the high GPA she has always wanted.

3. A friend needs help with studying (Darren, 22, 3rd-year college)

Darren is a student at SFSU and is very passionate about his studies. His friend, who attends SJSU, has been struggling with their classes lately and has contacted Darren for help. They are both Computer Science majors, so his friend asks him if he has any tips. Darren wants to send his friend a list of study resources to help them improve their grades. He tells his friend he uses a website called "StudentSources," which has a vast collection of resources for various subjects. Darren uses this site to study, so he sends his friend a link to his list of study resources. The resources are well-organized and easily accessible, which makes it easier for his friend to stay focused and motivated while studying. Darren also creates a group chat on the website with his friend and some of Darren's fellow Computer Science classmates. This way they can create a study group together and hopefully be more motivated to study for tests. With the help of the resources, Darren's friend can improve their grades and pass their classes. Darren is happy to have been able to help his friend; he and his friend continue to use StudentSources to stay on top of their studies.

4. Helping students (Dave, 42, Professor of Biology)

Dave is a professor of Biology at CCSF. He enjoys teaching his students and wants to ensure they have all the necessary resources to succeed in his class. One day, he hears about a website called StudentSources that allows him to create a list of study resources that his students can access. He makes a list on StudentSources for his class, including helpful slides, textbook pages, and practice exam PDFs. Dave shares this list with his students at the beginning of the semester, and they find it incredibly useful. They appreciate how organized and easy to use the website is and use it to stay on track throughout the semester. Some students also find other valuable notes uploaded by students for their other classes. Thanks to Dave's uploaded list of resources and the help of StudentSources, his students perform well in his class and feel confident in their understanding of the subject matter. Dave is happy to have found such a helpful tool and looks forward to using it for future classes.

5. Help for tutoring (Milena, 32, Tutor)

Milena is a dedicated tutor who wants to help her students succeed. She tutors middle schoolers mainly with subjects related to math. She has noticed that some of her students need help finding the right study resources to help them with their coursework.

To make things easier for her students, Milena has created a list of study resources using StudentSources that she can share with them. She understands that each student has a unique learning style and needs, so she has made sure to include a variety of resources that cater to different types of learners, which is made possible by the StudentSources website. She also has assigned group chats for students so that the students that are doing well in the class can chat with and hopefully help those falling behind. Milena's list comprises online resources, study guides, and practice problems specifically tailored to the subjects her students are studying. Her students are grateful for her help and find the resources helpful in improving their grades. Thanks to Milena's help and StudentSources, her students can achieve academic success and feel more confident in their learning abilities.

6. Technology Troubles (Marc, 20, organized student)

Marc is a scholar student at UC Berkeley who has been acing his class since the first semester. He has been attending classes regularly and making detailed notes of each lecture; moreover, every day at night, he regularly reviews his class notes and researches more on particular topics to gain valuable insights. All this work makes it easy for him to ace the finals since he has to revise his notes. However, two weeks before finals, his laptop fell down the table accidentally and broke along with all of his notes for the current and previous semesters. He went quickly to the service center for repair; however, they said it was impossible to fetch the data. Due to this, he could not study well or perform in the exams. The following semester, he came across StudentSources, which he used to store all his notes. He was happy to organize different links to various videos and articles related to his studies. This helped him stay focused and made him tension free from the laptop crash incident again since he would have a backup of all of his notes.

7. Attending many classes (Zander, 21, college student)

Zander is in a hurry to finish his graduation and has decided to take up all the remaining classes this semester only. He takes six courses for the semester, all of which are study intensive. After two weeks, he realizes he cannot keep up with the assignments and classes because the subjects and content must be clarified. It is easy to focus in class, but when he goes home and tries to study or work on assignments, he finds it hard to manage different notes and content. He comes across StudentSources, where he can create a list and content categorized by various factors like class name, professor, Etc. He finds it very straightforward to work on it and can manage his assignments well and on time. Moreover, over the semester break, he also adds data from his previous

semester. Whenever he is stuck on any advanced course, he can quickly check on the content of the related beginner courses and promptly clear his doubts. Hence, using StudentCourses, he handled the workload very effectively.

8. Landing A Job (Donovan, 26, recent graduate, looking for jobs)

Donovan recently graduated from SFSU and is now applying for jobs in various companies. However, due to the recession and the competitive market scenario, there are very few jobs in his desired field. This has made him very anxious since he is also pressured to repay education loan EMIs. While looking for jobs, he finds out that there are many openings for cybersecurity professionals. The problem is that Donovan last learned about cyber security in his bachelor's. Still, the good news is he used StudentCourses throughout his education to make notes and organize lists of the content. Searching through the course name, he found all the letters and content he used to learn cybersecurity. He was able to re-learn very quickly and crack a job interview as a cybersecurity specialist. This made him very happy and relieved as he could repay his education loan.

9. Overwhelming Course Load Candace — (26, overwhelming course load)

Candace is a Nursing major that has just transferred to SFSU from a community college and realized that the workload at the university level is much higher and finds it overwhelming. The semester begins, and she struggles to keep up with her classes. She considered withdrawing from three of her courses. She then met a classmate taking 24 units, and to her surprise, she "asked how she stays on task with her work?" She responds, "StudentSources, a website that helps organize a list of student resources." They converse more and start up a group chat in StudentSources. Candace and the classmate tell others to join StudentSources if they would like to be a part of the group chat and study together. Candace began using StudentSources and was able to catch up and ended up getting a 3.7 GPA in her first semester as a transfer, and also made some new Nursing major friends along the way.

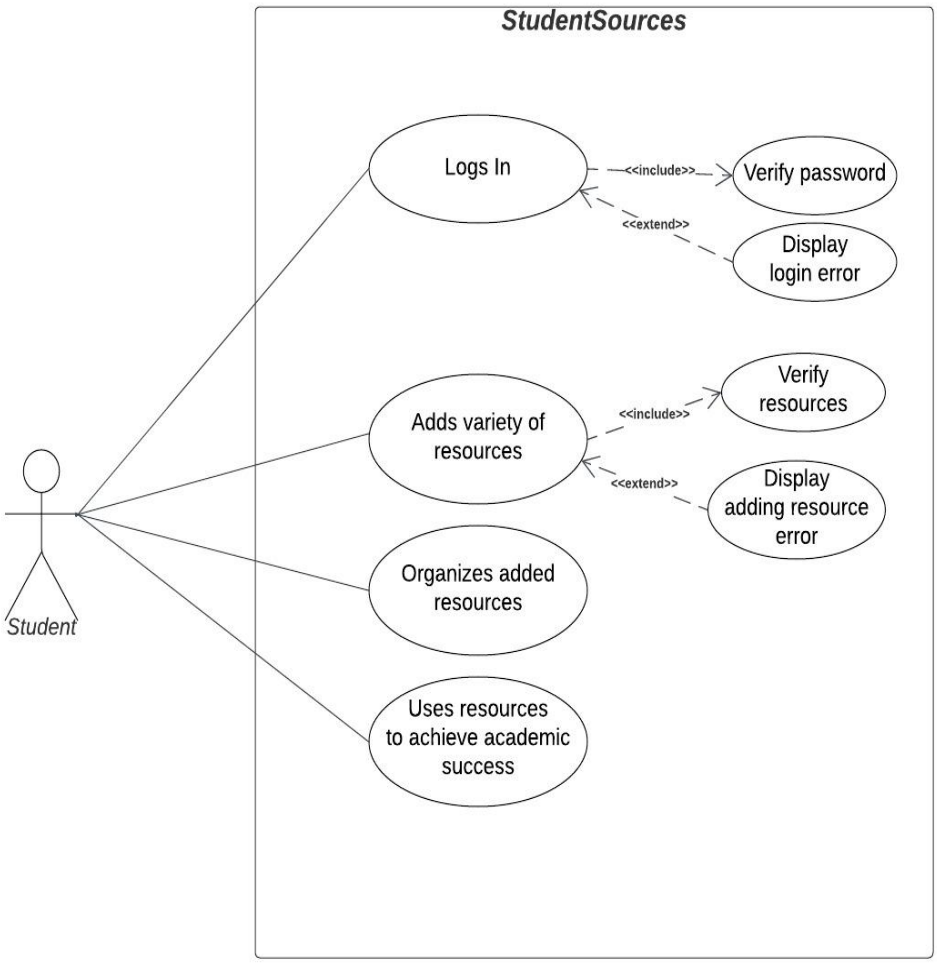
10. Imposter Syndrome (20, considering dropping out or changing majors)

Jacob is 20 yrs old and is a transfer student from Austin, Texas. He is in his first semester and has difficulty learning and feels that the lectures are going too fast for him. He was not able to record proper notes due to the speed of the professor changing the slides. This resulted in him considering dropping out or changing majors. After speaking with some of his fellow classmates, they introduced him to StudentSources, a

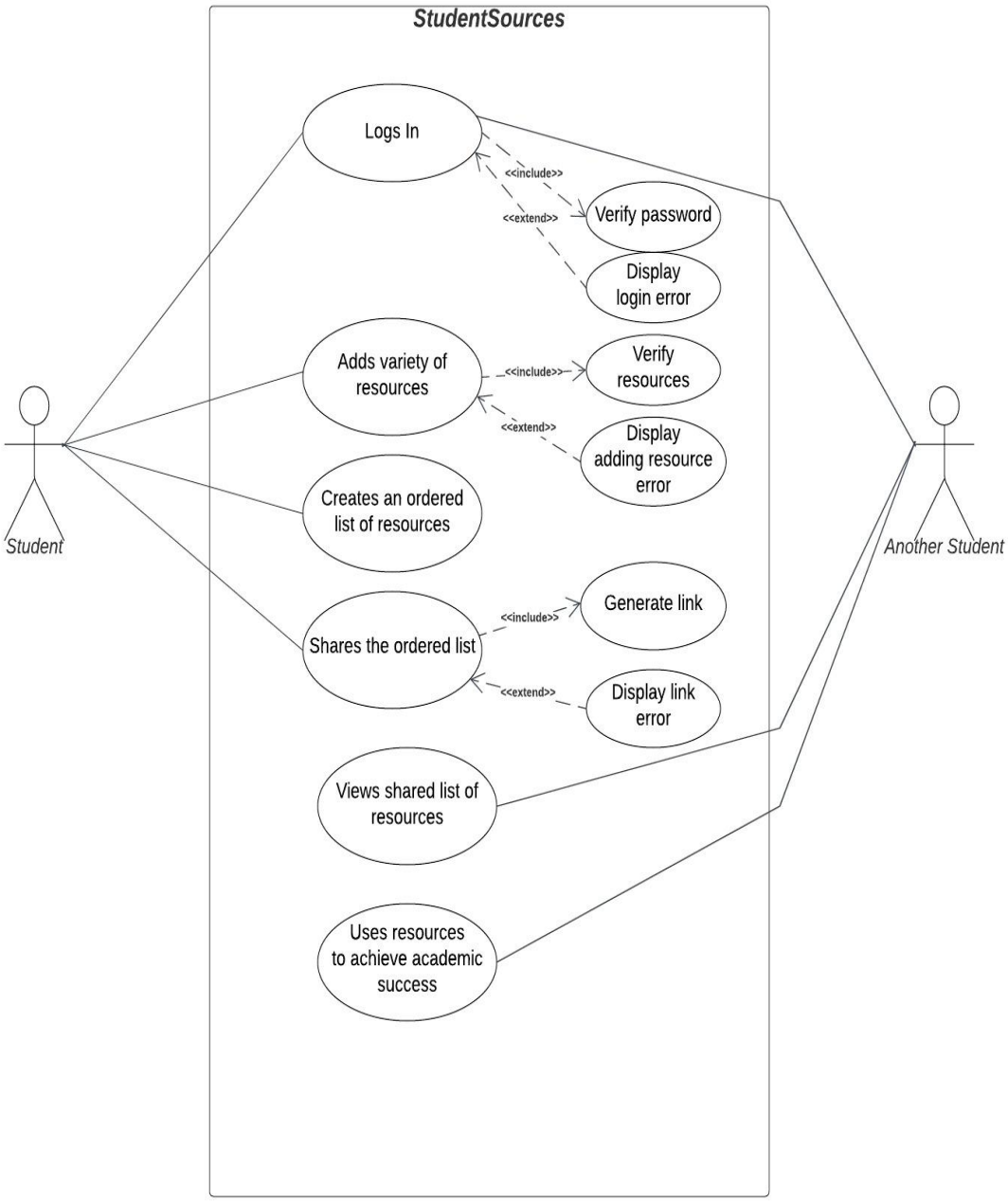
web page that gives him access to lists of organized resource links for students to improve performance. He starts using the website a lot, and his confidence in passing the class goes up. Using the features like comments and direct messaging, he lets the people who uploaded resources know that they really helped change how he takes and organizes his notes. Jacob does not need to worry about writing everything down because, with the help of StudentSources, he has learned so much more. Jacob decides to continue with his chosen field and is happy with where he is now.

Main Use Cases Diagram

A student organizes his online resources on StudentSources for their academic success.



A student shares his online resources on StudentSources to other students.



Main Data Items & Entities

1. **General User -** A general user can not view *approved resources*. General users do not need to log in/register.

2. **Registered User -** Registered users are attached to an *account*. A registered user can view *approved resources*. A registered user can upload *resources*. A registered user can open their files on a separate *tab*. A registered user must be logged in and registered.

3. **Account -** An account belongs to a *registered user*. An account stores user-related data such as username, real name, and posted resources.

4. **Admin -** An admin manages the website. An admin can access all data and content and modify the database, and verify resources.

5. **Resources -** Resources are the contents added by the users in their account such as text files, images/video, online contents.

6. **Unapproved Resource -** A resource is a specified data grouping that contains useful information for students. Resources can only be viewed by an *admin*.

7. **Approved Resource -** An approved resource is a *resource* that has been approved by an *admin*. Approved resources can be viewed by *admins* and *registered users*.

8. **Directory and Directory Tree -** A directory is a folder that contains resources inside. A directory tree is the structure to organize the contents in the directory.

9. **Comment -** Comment(s) are written remarks left by registered users on their own resource or other users' resources.

10. **Friend -** Friend(s) are other registered users who are added as 'Friend' by a registered user.

11. **Note -** Note(s) are the record of brief description, and thoughts that registered users can add to their account.

Functional Requirements

1. General User

1.1. A general user shall be able to create one or more accounts.
1.2. A general user shall be able to sign up with a unique email address.
1.3. A general user shall be able to sign up with a non-unique password.
1.4. A general user shall be able to see if all requirements for creating a password are met.
1.5. A general user shall be able to sign up with a non-unique username.
1.6. A general user shall be able to read the terms and conditions.
1.7. A general user shall be able to agree to the terms and conditions.
1.8. A general user shall be able to resend an account verification email.
1.9. A general user shall be provided a user manual covering basic features.

2. Registered User

2.1. A registered user shall have at least one account.
2.2. A registered user shall be able to interact with a skippable tutorial sequence.
2.3. A registered user shall be able to view their account information in the Account page.
2.4. A registered user shall be able to temporarily unhide account password information in the *Account* page. By default, when a registered user visits the *Account* page, the password would be hidden for security and password.
2.5. A registered user shall be able to change their account username.
2.6. A registered user shall be able to change their account email.
2.7. A registered user shall be able to change their account password.
2.8. A registered user shall be able to upload any supported file type to their account's cloud storage.
2.9. A registered user shall be able to cancel an upload that is in progress.
2.10. A registered user shall be able to navigate through files with a directory tree.
2.11. A registered user shall be able to organize files alphabetically through a directory tree.
2.12. A registered user shall be able to view the current directory from a *Files* panel.
2.13. A registered user shall be able to create directories.
2.14. A registered user shall be able to create text files.
2.15. A registered user shall be able to create bookmarks of web URLs.

2.16. A registered user shall be able to pin the resources for easy access in the future.
2.17. A registered user shall be able to view supported files on the website.
2.18. A registered user can view the contents of their files on a *Display* panel.
2.19. A registered user can view the contents of their saved web pages on a *Display* panel.
2.20. A registered user shall be able to change the color theme of the website.
2.21. A registered user shall have a place to type and save a note for each resource.
2.22. A registered user may have an assistant AI chatbot.
2.23. A registered user shall be able to highlight text on pdf files.
2.24. A registered user shall be able to comment on files.
2.25. A registered user shall be able to delete/move resources to a trash bin.
2.26. A registered user shall be able to delete resources from a trash bin permanently.
2.27. A registered user shall be able to recover items from a trash bin.
2.28. A registered user shall be able to download files onto their computer.
2.29. A registered user shall be able to share their directories via generated link.
2.30. A registered user shall be able to share their directories by configuring the list as ‘public’.
2.31. A registered user shall be able to configure permissions of their shared directories/folders. The permission allows other users to either only view the list or download the list.
2.32. A registered user shall be able to view other directories shared via link.
2.33. A registered user shall be able to view other directories shared publicly.
2.34. A registered user shall be able to download other shared directories into their account.
2.35. A registered user shall be able to download other shared directories onto their computer.
2.36. A registered user shall be able to search public directories.
2.37. A registered user shall be banned for uploading one or many inappropriate resources by admin.
2.38. A registered user shall not be able to view unapproved resources.
2.39. A registered user shall not be able to share unapproved resources.
2.40. A registered user shall be able to add other registered users as friends.
2.41. A registered user shall be able to remove other registered users from friend-list.
2.42. A registered user shall be able to mark other registered users as their favorite friend.
2.43. A registered user shall be able to edit their posted comments at any time.
2.44. A registered user shall be able to edit their saved notes at any time.

2.45. A registered user shall be able to configure visibility of their resources at any time. The visibility can be either public or private.
2.46. A registered user shall be able to message another registered user.
2.47. A registered user shall be able to create a group chat to message multiple other registered users.
2.48. A registered user shall be able to search other registered users whose profile is set public.
2.49. A registered user shall be able to search friends.
2.50. A registered user shall be able to search pinned resources.
2.51. A registered user shall be able to clear notes added to the resource.
2.52. A registered user shall be able to clear comments added to the resource.
2.53. A registered user shall be able to disable comments in the resource.
2.54. A registered user shall be able to delete a group chat they created.
2.55. A registered user shall be able to rename their resources.

3. Account

3.1. Account shall have a certain amount of cloud storage.
3.2. Accounts shall require and store a username that is at least 6 characters.
3.3. Accounts shall require and store a real name tied to them, and the real name must have a first and last name.
3.4. Accounts shall require and store a password tied to them.
3.5. Accounts password shall be encrypted when stored in a database.
3.6. Accounts shall be created by one or more general users.
3.7. An account shall be owned by one registered user.

4. Admin

4.1. Admin shall be able to access all resources, approved or not
4.2. Admin shall be able to delete any resources present on the website
4.3. Admin shall be able to change an unapproved resource to an approved resource.
4.4. Admin shall be able to change approved resources to unapproved resources.
4.5. Admin shall be able to delete/ban any user for any reason (breaking of rules/TOS, inappropriate username)
4.6. Admin shall be able to delete any inappropriate resources uploaded.
4.7. Admin shall be able to view zero or many unapproved resources.
4.8. Admin shall check if comments on resources are appropriate.
4.9. Admin shall delete zero or many inappropriate comments on one or many resources.

5. Resources

5.1. Resources shall be uploaded by a registered user.
5.2. Resources shall be organized in an alphabetical order.
5.3. Resources such as text files shall be able to be created by a registered user.
5.4. Resources shall be viewed by a registered user.
5.5. Resources shall be commented by registered users.
5.6. Resources shall be moved by a registered user to a trash bin.
5.7. Resources shall be able to be deleted permanently by a registered user from a trash bin.
5.8. Resources shall be recovered by a registered user from a trash bin.
5.9. Resources shall be downloaded on the computer by a registered user.
5.10. Resources shall have comments from other registered users.
5.11. Resources shall be able to be accessed by admin.
5.12. Resources shall be able to be deleted by admin.
5.13. Resources shall be deleted if deemed inappropriate by the admin.
5.14. Resources such as pdf files shall be highlighted by registered users.
5.15. Resources shall have comments verified by admin.
5.16. Resources shall be pinned by a registered user.
5.17. A resource shall have a note created by a registered user.
5.18. Resources shall have configured visibility by a registered user.
5.19. A pinned resource shall be searched by registered users.
5.20. Resources shall be able to be renamed by registered users who added them.

6. Unapproved Resource

6.1. An unapproved resource shall only be viewed by an admin.
6.2. An unapproved resource shall not be viewable by users until approved.
6.3. An unapproved resource shall be able to be converted to an approved resource by admin.
6.4. Unapproved resources shall not be able to be shared until an admin approves it.

7. Approved Resource

7.1. Approved resources shall be able to be viewed by admins and users.
7.2. Approved resources shall be able to be converted back to unapproved resources by the admin.
7.3. Approved resources shall be able to be shared via a generated link.

8. Directory and Directory Tree

8.1. Directory trees shall be navigated by a registered user.
8.2. Directory tree shall have files organized by a registered user.

8.3. Directories shall be created by a registered user.
8.4. Directories shall be able to be shared by registered users to other registered users via generated link.
8.5. Directories shall be able to be shared by registered users by configuring them to be ‘public.’
8.6. Directories shall have configured permissions added by a registered user.
8.7. Directories shared via generated links shall be able to be viewed by a registered user.
8.8. Directories shared by configuring it to ‘public’ shall be able to be viewed by a registered user.
8.9. Directories shall be able to be downloaded by other registered users onto their own account.
8.10. Directories shall be able to be downloaded by other registered users onto their own computer.
8.11. Directories of other users registered configured as public shall be able to be searched by registered users.

9. Comment

9.1. Comments shall be added on resources by registered users.
9.2. Comments on resources shall be verified by admin.
9.3. Comments shall be deleted by the admin, if deemed inappropriate.
9.4. Comments shall be able to be edited at any time by the registered user who posted it.
9.5. Comments shall be able to be cleared by a registered user at any time.
9.6. Comments shall be disabled by a registered user at any time.

10. Friend

10.1. A friend shall be added by registered users.
10.2. A friend shall be removed by registered users.
10.3. A friend shall be added as a favorite by registered users.
10.4. A friend shall be searched by registered users.

11. Note

11.1. A note shall be typed and saved by a registered user for each resource.
11.2. A note shall be able to be edited at any time by the registered user that saved it.
11.3. Notes shall be cleared by registered users at any time.

Non-Functional Requirements

1. System Requirements

1.1. System shall always hide registered users account password information on startup the *Account* page.
1.2. System shall periodically delete files from a trash bin.
1.3. System shall provide information to registered users the amount of free storage remaining.
1.4. System shall display an error message when a registered user exceeds the cloud storage limit for their account.
1.5. System shall show a registered user a progress and estimated wait time upon uploading files.

2. Performance

2.1. Any response to StudentSources links shall be within 2 seconds.
2.2. Any response to any StudentSources links shall be within 4 seconds during high volume
2.3. The website shall be able to load text and image files with large sizes within 3 seconds and audio and video with large sizes within 10-15 seconds.
2.4. The website shall be able to load text and image files with large sizes within 5 seconds and audio and video with large sizes within 20 seconds during high volume.
2.5. Log-in and logout of websites shall be almost instant.
2.6. Buttons, textboxes, and options on the landing UI shall be responsive.
2.7. Buttons, textboxes, and options on upload content UI shall be responsive.
2.8. StudentSources shall be able to handle a large number of simultaneously active users.
2.9. Uploaded audio and linked audio and video files shall be streamed to promote better user experience.

3. Storage

3.1. StudentSources shall be able to handle files with a large size, with the max size being 1 GB.
3.2. The cloud storage container shall properly store files in TXT format.
3.3. The cloud storage container shall properly store files in PDF format.
3.4. The cloud storage container shall properly store files in MP3 format.
3.5. The cloud storage container shall properly store files in URL format.
3.6. The cloud storage container shall properly store files in JPEG format.
3.7. The cloud storage container shall properly store files in JPG format.

3.8. The cloud storage container shall properly store files in PNG format.
3.9. The cloud storage container shall properly store files in GIF format.
3.10. The cloud storage container shall be able to handle different file formats while still being efficient.
3.11. The database and cloud storage shall store metadata associated with files stored in the cloud storage container.

4. Security

4.1. Users shall have a registered account to have access to all StudentSources files.
4.2. Passwords shall be encrypted in the database.
4.3. The website shall use secure hosting (AWS).
4.4. The website shall ensure only registered users can access resources.
4.5. The website shall ensure only admins can access sensitive information.

5. Environmental

5.1. The website shall be optimized for energy efficiency, optimizing images, using efficient code.
5.2. StudentSources shall encourage students to use electronic files instead of printing and using materials to reduce paper waste and the environmental impact of printing.
5.3. StudentSources shall encourage responsible use by prompting users to delete unwanted files when max capacity of the account is reached to reduce storage and bandwidth of the website.

6. Marketing

6.1. StudentSources shall be marketed primarily to students.
6.2. The website shall be marketed as a better way for students to store and organize study resources.
6.3. The website will be marketed as a way to reduce paper waste.

7. Legal

7.1. StudentSources shall comply with copyright laws, to ensure that when the website stores files without infringing the copyrights of others.
7.2. StudentSources shall obtain permission to use copyrighted material according to fair use and copyright laws.
7.3. StudentSources shall have a privacy policy explaining how personal data is collected, used, and protected, which is important for things like student information.
7.4. StudentSources shall have security measures to prevent unauthorized access of data stored in the website's database.

7.5. StudentSources shall have terms and conditions that govern the use of the website by users.
7.6. Terms and Conditions shall explain rules about uploading and downloading files and acceptable use of the site.

8. Content

8.1. Content present on the website shall be mostly if not all related to academic courses.

9. Look and Feel

9.1. StudentSources' landing page UI shall be simple for users to understand.
9.2. The website shall be easy to navigate, with users only spending no more than 3-5 minutes browsing around.
9.3. StudentSources UI shall utilize non-distracting colors that complement the content.
9.4. StudentSources UI for uploading content shall be easy to understand and navigate.
9.5. The website shall have a clean and minimalist design,avoiding clutter and aiming for a sleek appearance.

10. Usability

10.1. The website shall have no issues running when experiencing high volume.
10.2. The website shall maximize user experience in storing and organizing study resources.

11. Privacy

11.1. StudentSources shall only collect data that is necessary for the website's purpose.
11.2. Passwords shall be encrypted to ensure data security.

12. Compatibility

12.1. The website shall be accessible using branded computers such as Windows, Apple, Asus, etc.
12.2. The website shall be accessible on a range of devices like desktop computers, laptops, tablets, smartphones.
12.3. Website shall be compatible with operating systems such as Windows and macOS.
12.4. Website shall be compatible with a wide range of web browsers, including but not limited to: Chrome, Firefox, Edge, Safari.
12.5. The website shall support uploading of a range of file formats, like TXT, PDF, MP3, URL, JPEG, JPG, PNG, GIF.

13. Organizational

13.1. StudentSources shall have a user-centered design, focused on ensuring the website is easy to use and navigate for users.

13.2. The website shall have a clear purpose for students and stakeholders and not be too broad.

14. Coding Standards

14.1. The source code shall be clear.

14.2. Source code shall be concise.

14.3. Source code shall be consistent.

14.4. Source code shall be well commented.

14.5. Source code file names shall be clear.

14.6. Source code file names shall be concise.

14.7. Source code file names shall be consistent.

14.8. The file structure of source code shall be easy to navigate.

Competitive Analysis

Important features of competitive products

Company Features	Padlet	Wakelet	Bublup	LiveBinders	Dropbox
Strengths	Customizable and collaborative.	Intuitive UI placement. Offers features for students of all levels	Lots of functionality	Good for sharing docs in a binder format.	Very fast upload and syncing.
Weaknesses	Unfamiliar user interface and limited cloud storage. Too costly for students.	Limited features.	Some features do not work as expected. Offers low storage.	Difficult for the average user. Storage is limited for all levels/tiers	Lacks themes, customization, and easy sharing.
Pricing	Free tier, $6.99/ month tier, and $9.99/ month tier.	Free tier, then requires a quote for upgrading.	Free tier, $6.99/ month tier, $9.99/ month tier, and $24.99/ month tier	Free tier, and $19.95/ month paid tier.	Provides freemium service with several plans. $9.99/ month - $24/ month plans
Social Media	Instagram.	Facebook, LinkedIn, Twitter, Instagram	Facebook, Pinterest, Instagram, YouTube, Reddit, TikTok.	Blogs, podcasts, YouTube, Twitter, Facebook, LinkedIn, and Instagram.	Twitter, Facebook, and YouTube.
Experience	Moderate learning curve with unfamiliar UI.	Easy to navigate, set up, and organize resources.	Provides plenty of instructions throughout the setup process.	Features are confusing to use.	Provides plenty of support and features in an intuitive manner.

Key Features of Competitors vs. StudentSources

Company Features	Padlet	Wakelet	Bublup	LiveBinders	Dropbox	StudentSources
UI Customization	+	-	+	+	-	++
Easy User Interface	++	+	+	-	+	++
Cloud Integration	-	+	-	+	+	+
Custom Collection Sharing	-	+	-	+	+	++
Group Collaboration With Comment Feed	+	-	+	-	-	++

+ Feature exists **++ Superior** **- Does not exist**

Competitive Analysis Summary

After conducting our competitive analysis, we can see that the market for storage of general resources is out there but is usually limited and locked behind an expensive paywall, which for a college student is not optimal. With more essential expenses to worry about, such as housing, tuition, and textbooks, students like us will look for more cost-friendly options. Since our target audience is mainly college students, our website is superior due to the website being geared towards students. Competitors such as DropBox or LiveBinders have cloud storage for files but are mostly used for general file storage; in StudentSources, only files related to college and specific classes will be present. This results in students being able to easily find and access valuable resources from students who have taken or are taking same or similar courses. Based on data compiled from other study websites, we have concluded that the website we plan to create, StudentSources, is a better alternative for students based on convenience, UX/UI, and low pricing.

High-level System Architecture and Technologies Used

Server Host: Amazon EC2 Dedicated Host, t2.micro 1vCPU 1 GB RAM
Operating System: Ubuntu 22.04 Server
Database: MySQL 8.0.28
Web Server: NGINX 1.18.0
Server-Side Language: Javascript and Java
Additional Technologies:

- Web Framework: React and Spring Boot
- Web Wireframe: Figma
- IDE: VSCode and IntelliJ
- Web Analytics: Amazon Kinesis Data Analytics
- Runtime Environment: Node.js and Java Runtime Environment
- File-System: Amazon Simple Storage Service (Amazon S3)

Milestone 2

Data Definitions

General User: A general user is one who doesn't have permission to view any of the approved resources. As they also automatically lack an account on StudentSources, general users do not necessarily need to either log in nor do they have to register and have access to basic features. However, if they choose to do so, general users can sign up by providing an email address, password, and to make sure that they agree to the terms and conditions.

Registered User: Registered users are essentially users who have access to an account on StudentSources. Unlike general users, registered users have the capability to view resources that have been approved as well as have the capability to upload their own. Also, registered users are able to open their own file via creating separate tabs as long as they are logged in and registered into the system.

Account: An account is the one thing that differs between general users and registered users. Accounts store various forms of data belonging to those who are registered users, such as usernames, passwords, full names, and resources that said users have posted. Also, those who have accounts have full access to all approved and unapproved resources and have the capability to search for different files to be added to the list and/or share with other users.

Admin: Admin is a person/ team who is in charge of managing the website. The responsibilities of the admin include maintaining and troubleshooting the website, implementing security protocols, ensuring efficient user experience, and updating the website's contents. Admin also has authoritative power to approve or disapprove resources that users add to the website.

Resources: Resources are the contents added by the users in their account. A registered user can also pin a resource for easy access to the resource in the future. The list of resources are text files, images/video, online contents. The maximum size of the resources a registered user shall upload would be 1 GB.

- **Text files:** Text files include files with extensions .pdf, .doc, and .txt.
- **Images:** Image content includes files with extension .jpg, .jpeg, .gif and .png.
- **Video:** Video contents include files with extensions .mp4, and .mov.
- **Audio:** Audio contents include files with extensions .mp3.
- **Online Contents:** Online contents include the online resources such as link to webpage, YouTube videos.

Unapproved Resources: Unapproved resources are the resources that are yet to be approved by the admin or the resources that failed to acquire permission from the admin to be added to the website. The criteria for resources to be approved include the relevancy of the content and if it is related to the course. The unapproved resources are listed as approved resources once they receive approval. Suppose the resources are unsuccessful in gaining permission. In that case, the resource is removed from the website, and the respective user of the resource is informed via the notification panel of the website.

Approved Resources: Approved resources are the resources that are approved based on the approval criteria. Once the resources are approved by the admin, the users can add the content to the list and share it with other users with the generated link.

Directory and Directory Tree: A directory is a folder that contains resources inside. A directory tree is the structure to organize the contents in the directory. Registered users can create multiple directories and subdirectories dedicated to different courses and topics. Registered users can use the default layout or create it for note organization. Registered users can label directories as professor's name, course name, or course abbreviation and number (CSC648, for example). Registered users can share the directories and subdirectories with other users with the generated link.

Comment: Comments are the remarks that users leave for uploaded resources. Only registered users shall be able to leave a comment on a resource. Registered users can leave one or multiple comments on their own resources, shared resources or publicly accessible resources. The posted comments are verified if appropriate by the admin. If comments are not appropriate, the admin can delete the comment from the resource.

Friend: Friend(s) are other registered users who are added as ‘Friend’ by a registered user. Adding other registered users as friends helps find users faster by applying filters while searching. A registered user has options to add/remove friends, and to list friends as ‘Favorite.’

Note: Note(s) are the record of brief descriptions and/or thoughts registered users can add to their accounts. A panel is available for registered users to add notes to their accounts. The associated note panel contains features where a registered user can type and save notes on the website.

Prioritized Functional Requirements

Priority 1

General User

1.1. A general user shall be able to create one or more accounts.
1.2. A general user shall be able to sign up with a unique email address.
1.3. A general user shall be able to sign up with a non-unique password.
1.4. A general user shall be able to see if all requirements for creating a password are met.
1.5. A general user shall be able to sign up with a non-unique username.
1.6. A general user shall be able to read the terms and conditions.
1.7. A general user shall be able to agree to the terms and conditions.

Registered User

2.1. A registered user shall have at least one account.
2.3. A registered user shall be able to view their account information in the Account page.
2.4. A registered user shall be able to temporarily unhide account password information in the *Account* page. By default, when a registered user visits the *Account* page, the password would be hidden for security and password.
2.5. A registered user shall be able to change their account username.
2.6. A registered user shall be able to change their account email.
2.7. A registered user shall be able to change their account password.
2.8. A registered user shall be able to upload any supported file type to their account's cloud storage.
2.10. A registered user shall be able to navigate through files with a directory tree.
2.11. A registered user shall be able to organize files alphabetically through a directory tree.
2.12. A registered user shall be able to view the current directory from a *Files* panel.
2.13. A registered user shall be able to create directories.
2.14. A registered user shall be able to create text files.
2.15. A registered user shall be able to create bookmarks of web URLs.
2.16. A registered user shall be able to pin the resources for easy access in the future.
2.17. A registered user shall be able to view supported files on the website.
2.18. A registered user can view the contents of their files on a *Display* panel.
2.19. A registered user can view the contents of their saved web pages on a *Display* panel.

2.21. A registered user shall have a place to type and save a note for each resource.
2.22. A registered user may have an assistant AI chatbot.
2.24. A registered user shall be able to comment on files.
2.25. A registered user shall be able to delete/move resources to a trash bin.
2.26. A registered user shall be able to delete resources from a trash bin permanently.
2.27. A registered user shall be able to recover items from a trash bin.
2.28. A registered user shall be able to download files onto their computer.
2.30. A registered user shall be able to share their directories by configuring the list as ‘public’.
2.31. A registered user shall be able to configure permissions of their shared directories/folders. The permission allows other users to either only view the list or download the list.
2.33. A registered user shall be able to view other directories shared publicly.
2.35. A registered user shall be able to download other shared directories onto their computer.
2.36. A registered user shall be able to search public directories.
2.40. A registered user shall be able to add other registered users as friends.
2.41. A registered user shall be able to remove other registered users from friend-list.
2.42. A registered user shall be able to mark other registered users as their favorite friend.
2.45. A registered user shall be able to configure visibility of their resources at any time. The visibility can be either public or private.
2.46. A registered user shall be able to message another registered user.
2.47. A registered user shall be able to create a group chat to message multiple other registered users.
2.48. A registered user shall be able to search other registered users whose profile is set public.
2.49. A registered user shall be able to search friends.
2.50. A registered user shall be able to search pinned resources.
2.51. A registered user shall be able to clear notes added to the resource.
2.52. A registered user shall be able to clear comments added to the resource.
2.55. A registered user shall be able to rename their resources.

<u>Account</u>

3.1. Account shall have a certain amount of cloud storage.
3.2. Accounts shall require and store a username that is at least 6 characters.

3.3. Accounts shall require and store a real name tied to them, and the real name must have a first and last name.
3.4. Accounts shall require and store a password tied to them.
3.5. Accounts password shall be encrypted when stored in a database.
3.6. Accounts shall be created by one or more general users.
3.7. An account shall be owned by one registered user.

Admin

4.1. Admin shall be able to access all resources, approved or not
4.2. Admin shall be able to delete any resources present on the website
4.5. Admin shall be able to delete/ban any user for any reason (breaking of rules/TOS, inappropriate username)
4.6. Admin shall be able to delete any inappropriate resources uploaded.
4.8. Admin shall check if comments on resources are appropriate.
4.9. Admin shall delete zero or many inappropriate comments on one or many resources.

Resources

5.1. Resources shall be uploaded by a registered user.
5.2. Resources shall be organized in an alphabetical order.
5.3. Resources such as text files shall be able to be created by a registered user.
5.4. Resources shall be viewed by a registered user.
5.5. Resources shall be commented by registered users.
5.6. Resources shall be moved by a registered user to a trash bin.
5.7. Resources shall be able to be deleted permanently by a registered user from a trash bin.
5.8. Resources shall be recovered by a registered user from a trash bin.
5.9. Resources shall be downloaded on the computer by a registered user.
5.10. Resources shall have comments from other registered users.
5.11. Resources shall be able to be accessed by admin.
5.12. Resources shall be able to be deleted by admin.
5.13. Resources shall be deleted if deemed inappropriate by the admin.
5.15. Resources shall have comments verified by admin.
5.16. Resources shall be pinned by a registered user.
5.17. A resource shall have a note created by a registered user.
5.18. Resources shall have configured visibility by a registered user.
5.19. A pinned resource shall be searched by registered users.
5.20. Resources shall be able to be renamed by registered users who added them.

Directory and Directory Tree

8.1. Directory trees shall be navigated by a registered user.
8.2. Directory trees shall have files organized in alphabetical order.
8.3. Directories shall be created by a registered user.
8.5. Directories shall be able to be shared by registered users by configuring them to be 'public.'
8.6. Directories shall have configured permissions added by a registered user.
8.8 Directories shared by configuring it to 'public' shall be able to be viewed by a registered user.
8.10. Directories shall be able to be downloaded by other registered users onto their own computer.
8.11. Directories of other users registered configured as public shall be able to be searched by registered users.

Comment

9.1. Comments shall be added on resources by registered users.
9.2. Comments on resources shall be verified by admin.
9.3. Comments shall be deleted by the admin, if deemed inappropriate.

Friend

10.1. A friend shall be added by registered users.
10.2. A friend shall be removed by registered users.
10.3. A friend shall be added as a favorite by registered users.
10.4. A friend shall be searched by registered users.

Note

11.1 A note shall be typed and saved by a registered user for each resource.
11.3. Notes shall be cleared by registered users at any time.

Priority 2

General User

1.8. A general user shall be able to resend an account verification email.

Registered User

2.29. A registered user shall be able to share their directories via generated link.
2.32. A registered user shall be able to view other directories shared via link.

2.34. A registered user shall be able to download other shared directories into their account.
2.37. A registered user shall be banned for uploading one or many inappropriate resources by admin.
2.38. A registered user shall not be able to view unapproved resources.
2.39. A registered user shall not be able to share unapproved resources.
2.53. A registered user shall be able to disable comments in the resource.
2.54. A registered user shall be able to delete a group chat they created.

Admin

4.3. Admin shall be able to change an unapproved resource to an approved resource.
4.4. Admin shall be able to change approved resources to unapproved resources.
4.7. Admin shall be able to view unapproved resources.

Unapproved Resources

6.1. An unapproved resource shall only be viewed by an admin.
6.2. An unapproved resource shall not be viewable by users until approved.
6.3. An unapproved resource shall be able to be converted to an approved resource by admin.
6.4. Unapproved resources shall not be able to be shared until an admin approves it.

Approved Resources

7.1. Approved resources shall be able to be viewed by admins and users.
7.2. Approved resources shall be able to be converted back to unapproved resources by the admin.
7.3. Approved resources shall be able to be shared via a generated link.

Directory and Directory Tree

8.4. Directories shall be able to be shared by registered users to other registered users via generated link.
8.7. Directories shared via generated links shall be able to be viewed by a registered user.
8.9. Directories shall be able to be downloaded by other registered users onto their own account.

Comment

9.5. Comments shall be able to be cleared by a registered user at any time.

9.6. Comments shall be disabled by a registered user at any time.

Priority 3

General User

1.9. A general user shall be provided a user manual covering basic features.

Registered User

2.2. A registered user shall be able to interact with a skippable tutorial sequence.
2.9. A registered user shall be able to cancel an upload that is in progress.
2.20. A registered user shall be able to change the color theme of the website.
2.23. A registered user shall be able to highlight text on pdf files.
2.43. A registered user shall be able to edit their posted comments at any time.
2.44. A registered user shall be able to edit their saved notes at any time.

Resources

5.14. Resources such as pdf files shall be highlighted by registered users.

Comment

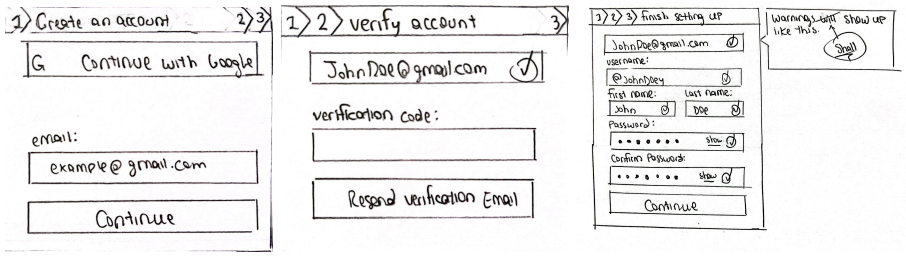
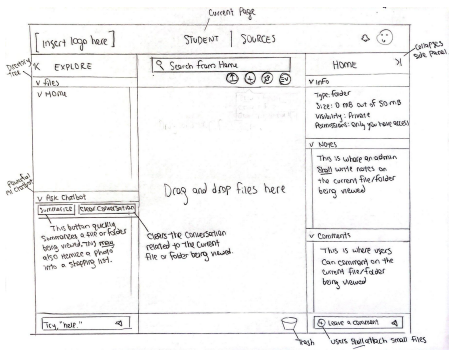
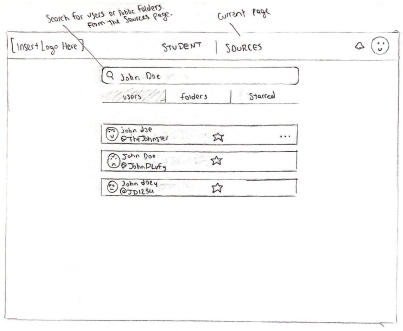
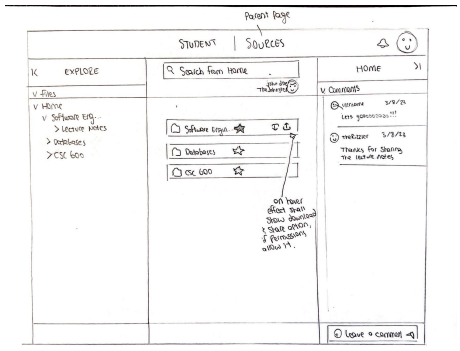
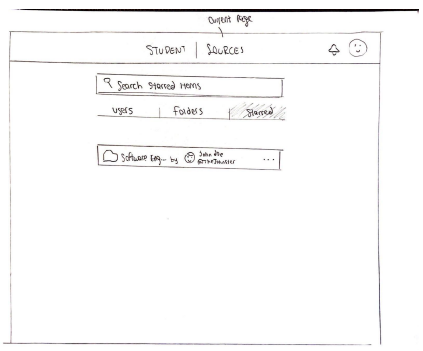
9.4. Comments shall be able to be edited at any time by the registered user who posted it.

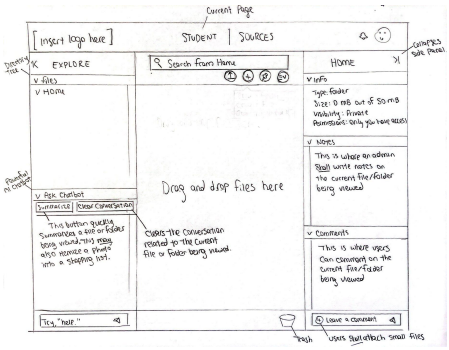
Note

11.2. A note shall be able to be edited at any time by the registered user who saved it.

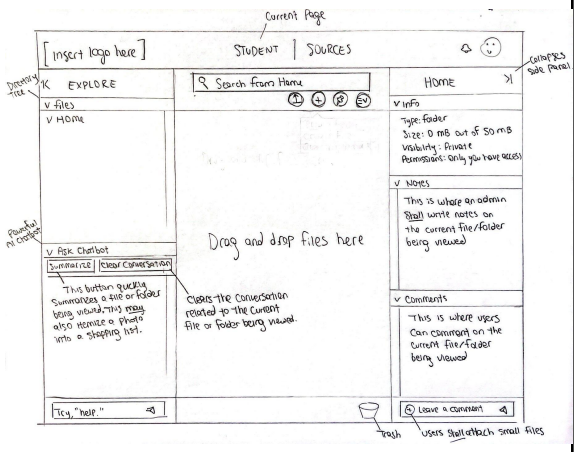
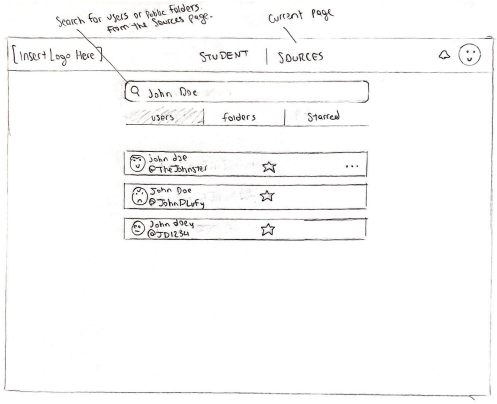
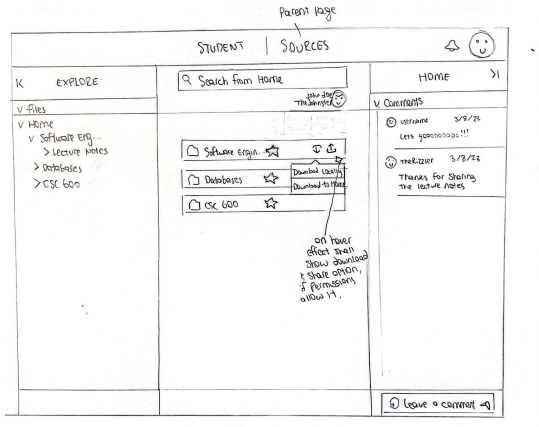
UI Mockups and Storyboards

Use case 1: Struggling with grades (Aaron, 19, 2nd year in college)

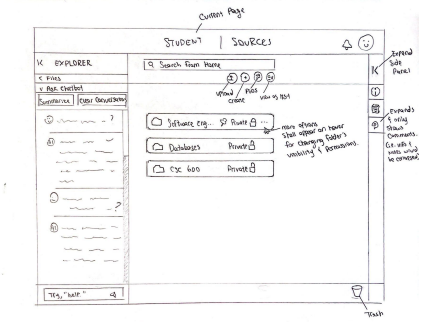
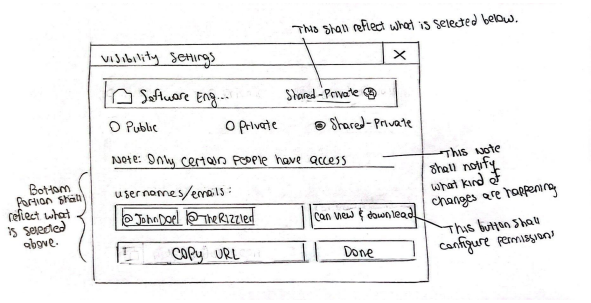
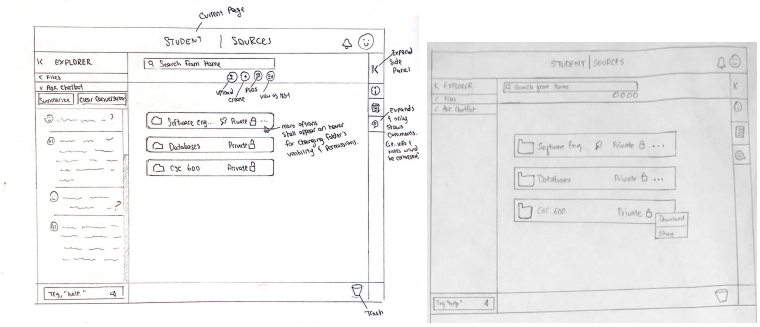
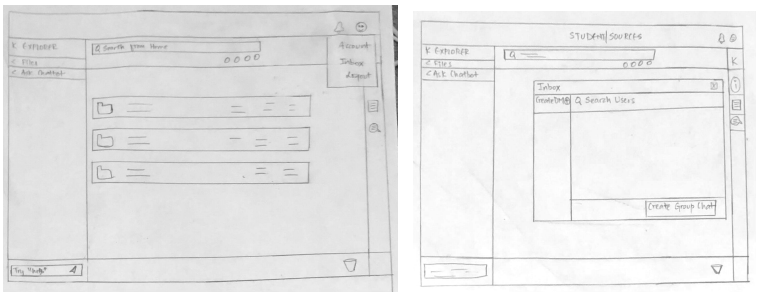
UI Mockup	Description
	Aaron goes to the signup page of StudentSources, where he registers an account. After he submits a valid email, he finishes setting up his account.
 	Aaron arrives at the landing/home page after signing in. Aaron goes to the sources tab to search for his friend who has the lecture notes for his software engineering course.
 	Aaron clicks on his friend's account, after which he is brought to his friend’s home page, where he can view all of his friend’s publicly made folders. Aaron leaves a star on the folder containing the lecture notes, so he can quickly access them from his starred list.

	After searching for his friend's account, Aaron also wants to try out the AI ChatBot feature. Aaron clicks on the Student tab then copies and pastes the notes on the Ask ChatBot panel, then clicks on the Summarize tab to summarize.

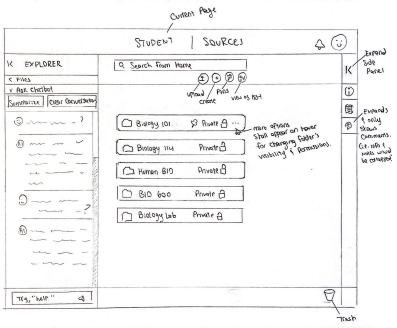
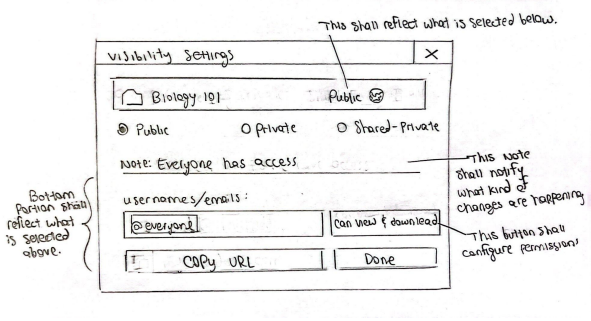
Use case 2: About to graduate (Sydney, 23, 4th-year college)

UI Mockup	Description
	Sydney arrives at her landing/home page after signing up and verifying her account. She wants to search for her friends' lecture notes, so she does not upload or create anything herself.
	Sydney navigates to the sources page to search for her friend, who was a previous student of a course Sydney is currently taking. The ‘users’ section is already auto selected upon arrival, so she begins searching from a list of StudentSources users.
	Sydney navigates to her friend’s home page by clicking on her username, and downloads the public folder containing relevant notes to her local computer, so that she can review them offline. Afterwards, she leaves a comment on her page to thank her friend for the lecture notes.

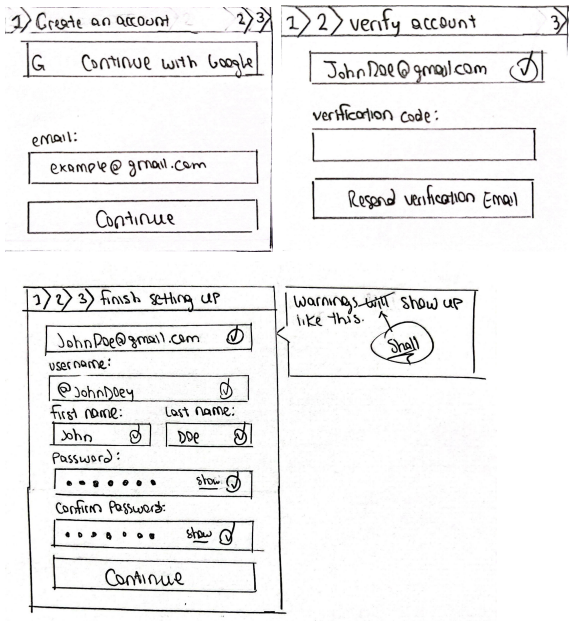
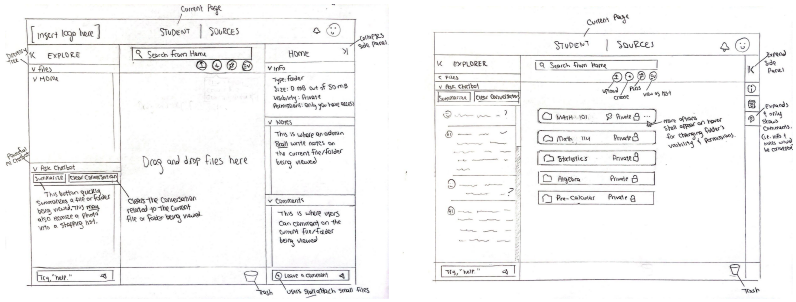
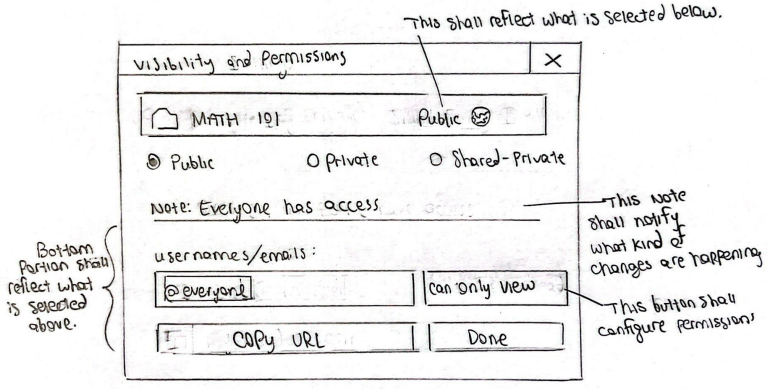
Use case 3: A friend needs help with studying (Darren, 22, 3rd-year college)

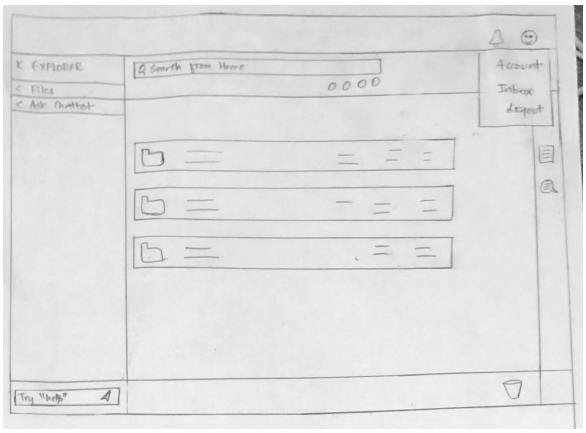
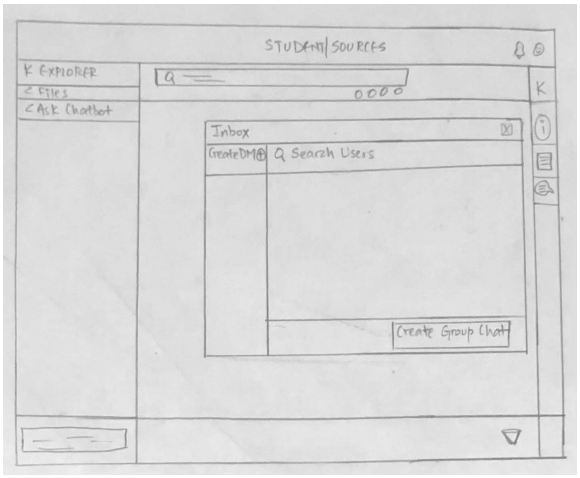
UI Mockup	Description
	Darren has a list of study materials for his Software Engineering class on the StudentSources website. He wants to keep his folder of resources private; not searchable on the sources page, yet he wants his friend to have access to it, and be able to download it.
	Darren configures his private folder to a shared-private folder in a pop-up window, where only a select few of valid StudentSources users have access to view and download his folder. With this feature, he can safely share his resources, while remaining anonymous from the general public.
	Darren generates a link to share his list by clicking on the Kebab menu on the folder which includes the Share option. After clicking on share, a generated link is copied to clipboard which Darren can save in any text file to use in the future.
	Darren creates a group chat with his friends to send his ordered list by clicking on the User icon on the top-right then on the Inbox option. There, he adds his friends to group chat.

Use case 4: Helping students (Dave, 42, Professor of Biology)

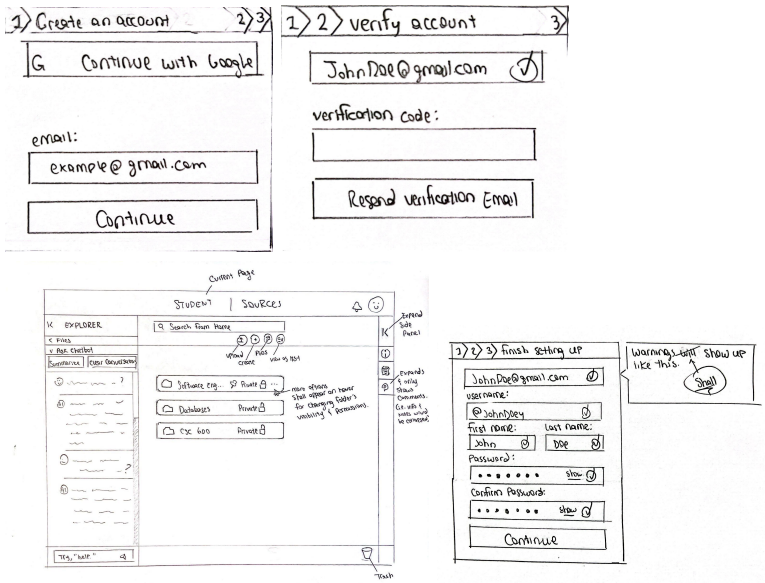
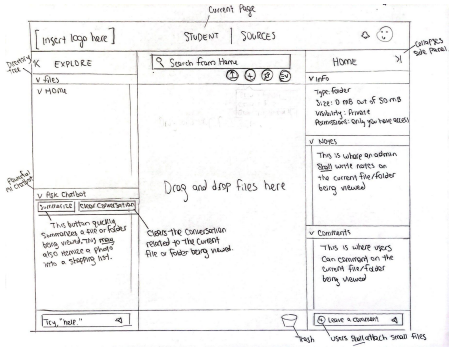
UI Mockup	Description
	Dave is a professor teaching several subjects in Biology, and wants to make it easy for his students to access his curated list of biological research and documentation.
	Dave sets his folders of various Biological subjects to ‘public’ visibility in a pop up window, because he wants everyone, including those who cannot afford to go to college, to easily view and download his carefully curated material.

Use case 5: Help for tutoring (Milena, 32, Tutor)

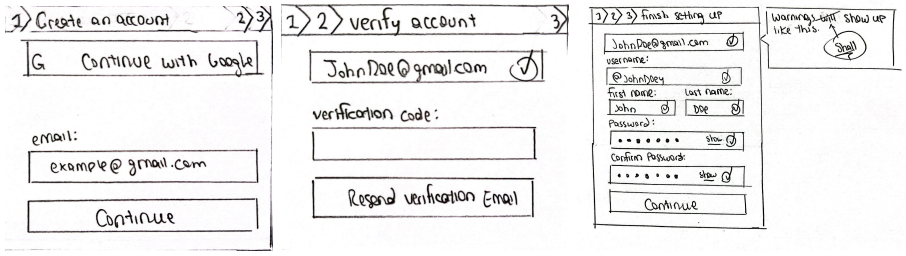
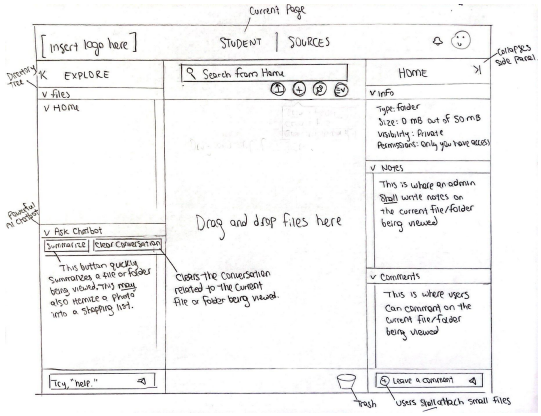
UI Mockup	Description
	Milena is a middle school tutor of various mathematical subjects, and wants to create a list of resources. She already has some curated folders of resources stored on her personal cloud account and wishes to make these folders publicly visible so that her students can access them. But she is uncomfortable with the idea of having middle schoolers accessing her personal cloud account. So she sets up an account on StudentSources to have the folder of study material publicly accessible to her students.
	Milena closes some panels before dragging and dropping some folders containing study material onto her home page, where it is automatically set to private. This means that nobody can see these files on the sources page, and that it has not been shared with anyone else.
	Milena sets the folder to public in a pop-up window named “visibility and permissions”, so that anybody can search it from the sources page, but sets the permissions to view-only, so that they cannot download it.

 	Milena also creates group chats for her students by clicking on the User icon then Inbox option. Then she adds her students to group chats.

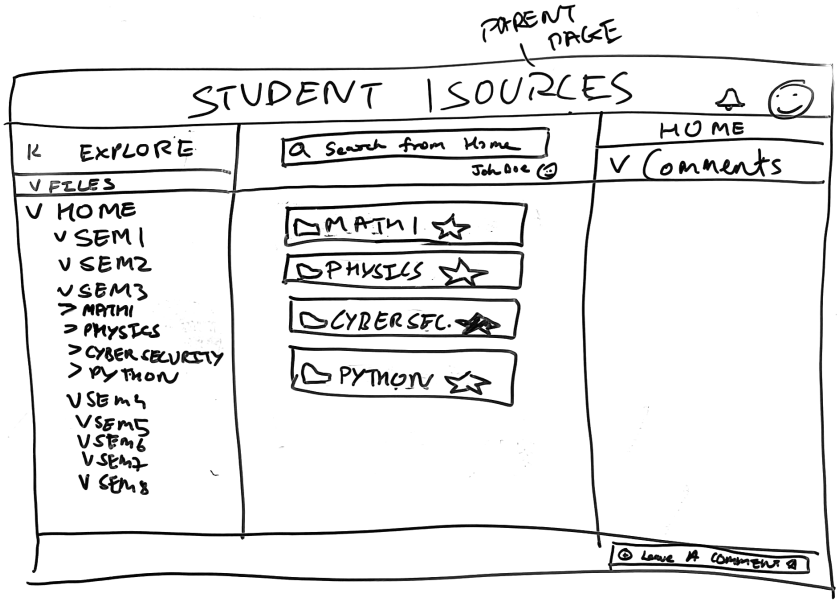
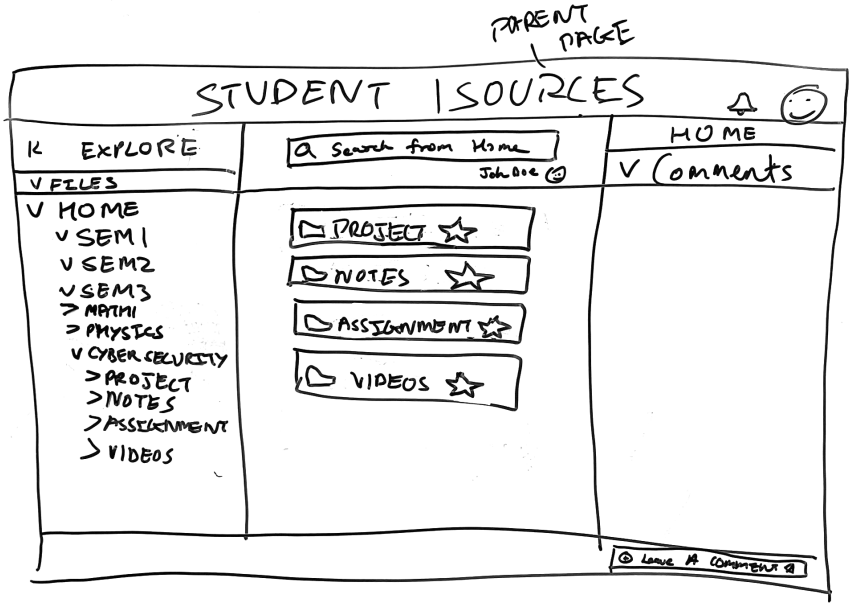
Use case 6: Technology Troubles (Marc, 20, Organized student)

UI Mockup	Description
	After losing all of the notes because of a broken laptop. For this semester Marc first goes to the signup page of StudentSources, where he registers an account. After he submits a valid email, he finishes setting up his account.
	Marc arrives at the landing/home page after signing in. Initially he has no notes or assignments at the start of the semester. He creates separate folders for each course by clicking on the create button and stores his notes here which he is able to access across different computers.

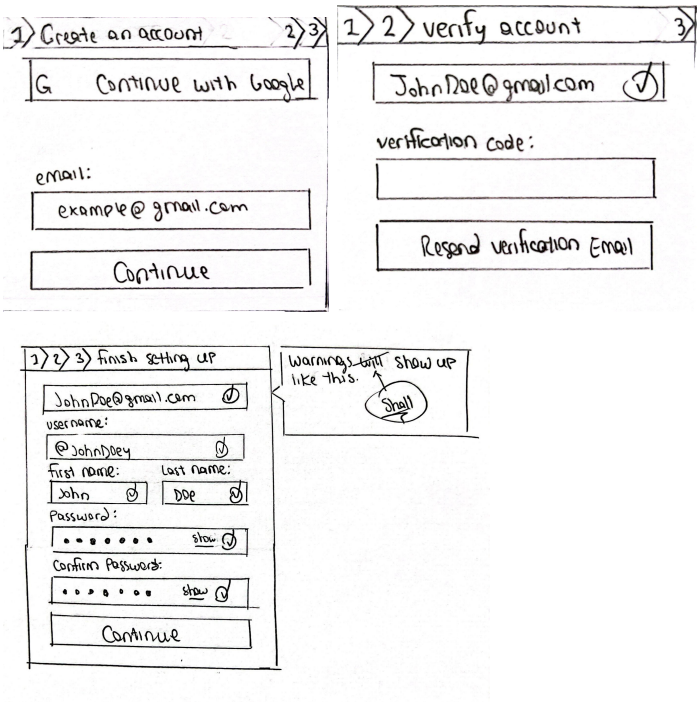
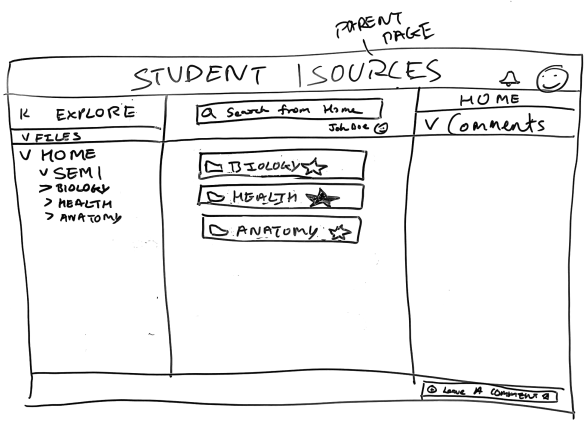
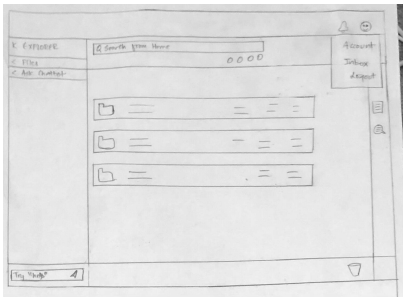
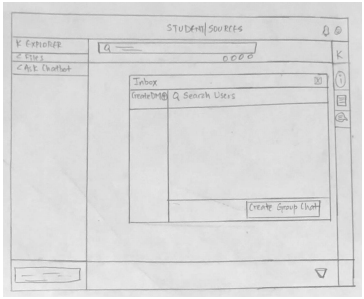
Use case 7: Attending many classes (Zander, 21, college student)

UI Mockup	Description
	Zander goes to the signup page of StudentSources, where he registers an account. After he submits a valid email, he finishes setting up his account.
	Zander, being overwhelmed by the courses, organizes the courses in a well manner with folders. First he starts creating a folder for each semester. Then in each semester folder he organizes different courses. Hence whenever he is stuck he can go back to the previous semester to learn again!

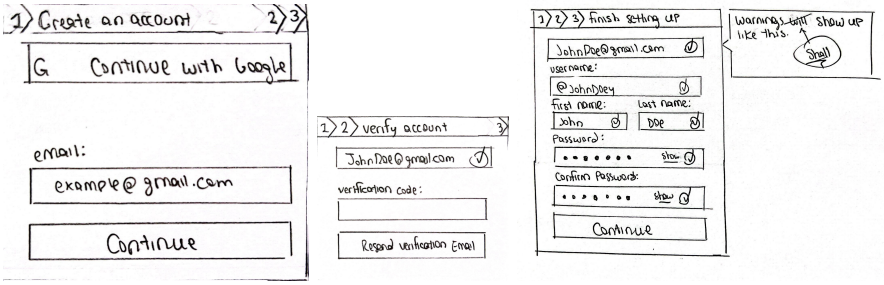
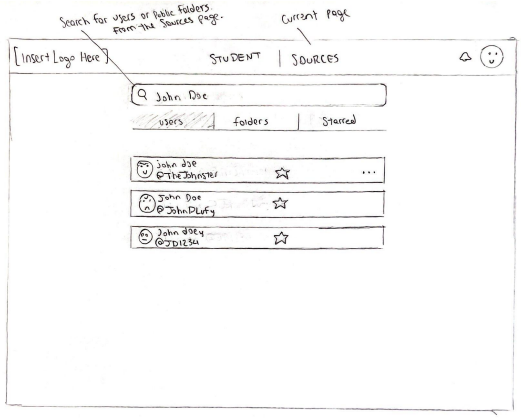
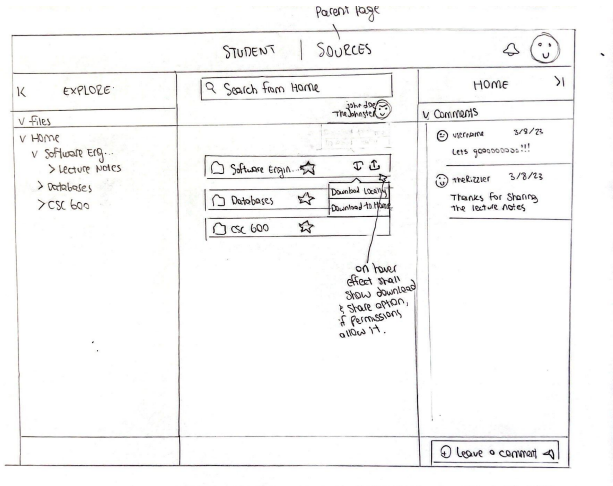
Use case 8: Landing A Job (Donovan, 26, recent graduate, looking for jobs)

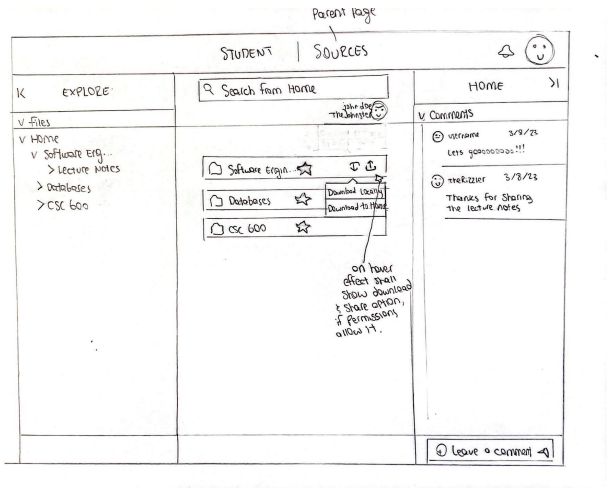
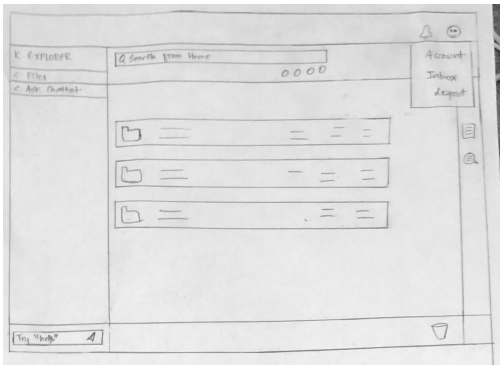
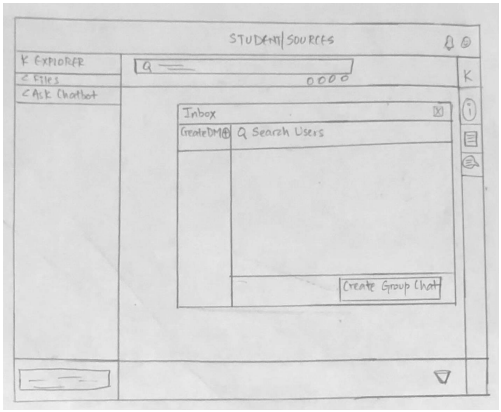
UI Mockup	Description
 	Donovan, is a recent SFSU graduate and is now looking for a job. He had the good habit of storing all of his notes in Student sources. So now he is looking for a job in the cyber security niche. He logins and goes to the cyber security folder to access all the project, notes, assignments and videos.

Use case 9: Overwhelming Course Load Candace — (26, overwhelming course load)

UI Mockup	Description
	Candace goes to the signup page of StudentSources, where she registers an account. After she submits a valid email, she finishes setting up her account.
	Candace, a nurse major & a transfer student. Uses student sources to store her notes for different subjects in an organized way and is able to keep up with classes and gets good GPA in her first semester
 	Candance creates a group chat and adds her classmates by clicking on the User icon then Inbox option.

Use case 10: Imposter Syndrome (20, considering dropping out or changing majors)

UI Mockup	Description
	Jacob goes to the signup page of StudentSources, where he registers an account. After he submits a valid email, he finishes setting up his account.
	Jacob is unable to keep up with the professor's speed to take up notes. He searches for a classmate's name and taps on his name.
	On clicking it opens up the friend's folder and he is able to access the public folder and is able to revise and study well.

	Jacob leaves comments on public directories by navigating to the Comments panel and typing his comment on a provided space.
 	Jacob also reaches out to other users by using the Direct Message feature. He clicks on the User icon on the top-right, then clicks on the Inbox option. In the new window he selects Create DM to message others.

High level database architecture and organization

This high level conceptual database design will be based on one of the unique features StudentSources provides, which is the exclusivity to SFSU students and being able to have a dedicated community for SFSU study resources.

Database Organization

Database Requirements:

1. A registered user shall be able to upload zero or many resources.
2. A registered user shall add zero or many comments on their own resources.
3. A registered user shall comment on many other registered users' resources.
4. A registered user shall add zero or many other registered users as friends.
5. A registered user shall remove zero or many other registered users from friend-list.
6. A registered user shall add zero or one note for each resource.
7. A registered user shall set permissions to ‘downloadable’ of many of their shared lists.
8. A registered user shall set permissions to ‘view only’ of many of their shared lists.
9. A registered user shall set visibility as ‘public’ of many of their resources.
10. A registered user shall set visibility as ‘private’ of many of their resources.

Entities:

1. registeredUser
2. userResource
3. resourceComment
4. userFriend
5. sharedList
6. userNote

Entity Attributes and Domains:

1. registeredUser
    a. userID (primary key) = INT, numeric, unique, and not null

    b. username = VARCHAR(32), not null, alphanumeric, can be upper and lowercase, must be unique

    c. email = VARCHAR(50), not null, alphanumeric, must contain valid email with a @

d. password = VARCHAR(32), not null, alphanumeric, will be encrypted before storing, must have requirements specified by website

e. formalName = VARCHAR(32), first and last name of user, can only contain letters

f. userImage = BLOB, user profile image saved as binary data

g. sharedListID (foreign key) = INT, numeric, references sharedList's sharedListID

h. resourceID (foreign key) = INT, numeric, references userResources' resourceID

i. commentID (foreign key) = INT, numeric, references resourceComment's commentID

j. noteID (foreign key) = INT, numeric, references userNote's noteID

2. userResource
    a. resourceID (primary key) = INT, numeric, unique, not null, auto incremental

    b. title = VARCHAR(32), not null, alphanumeric, must be unique to prevent confusion of resources.

    c. description = TEXT, can contain anything TEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 65,535 bytes.

    d. uploadDate = DATE, Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'

    e. courseID = VARCHAR(12), alphanumeric, will be stored as 3-4 letters following 3 numbers, for example: CSC101

    f. professorName = VARCHAR(32), alphanumeric, can only contain letters and hyphens

g. isPrivate = TINYINT(1), equivalent to boolean data type, can only contain 0 or 1, tracks if the resource is private or public

h. userID (foreign key) = INT, numeric, references registeredUser's userID

i. commentID (foreign key) = INT, numeric, references resourceComment's commentID

j. noteID (foreign key) = INT, numeric, references userNote's noteID

userResource entity contains child entities which define the format of resources that users store. The child entities with their respective attributes are as follows:

i. textFile
   a. size = INT, numeric
   b. extension = VARCHAR(10), alphanumeric, can contain letters and a special character like '.'

ii. image
   a. size = INT, numeric
   b. extension = VARCHAR(10), alphanumeric, can contain letters and a special character like '.'

iii. video
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

iv. audio
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

v. webpage
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

3. resourceComment
   a. commentID (primary key) = INT, numeric, not null, unique

   b. comment = TINYTEXT, alphanumeric, can contain anything TINYTEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 255 characters.

   c. commentDate = DATE, Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'

   d. resourceID (foreign key) = INT, numeric, references userResource's resourceID field, creating a relationship between the resource and the comment that was posted on the resource.

   e. userID (foreign key) = INT, numeric, references registeredUser's userID field, creating relationship between the comment and the user who commented it.

4. userFriend
   a. UID1 (primary key and foreign key) = INT, numeric, not null, references registeredUser's userID
   b. UID2 (primary key and foreign key) = INT, numeric, not null, references registeredUser's userID

5. sharedList
   a. sharedListID (primary key) = INT, numeric, not null

   b. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

   c. isDownloadable = TINYINT(1), equivalent to boolean data type, can only contain 0 or 1, tracks if the resource is view only or is downloadable by other registered users

   d. userID (foreign key) = INT, numeric, references registeredUser's userID

   e. resourceID (foreign key) = INT, numeric, references userResource's resourceID

6. userNote
   a. noteID (primary key) = INT, numeric, not null,

b. note = TINYTEXT, alphanumeric, can contain anything TINYTEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 255 characters.

c. userID (foreign key) = INT, numeric, references registeredUser's userID

d. resourceID (foreign key) = INT, numeric, references userResources' resourceID

Entity Relationships:

1. One-to-Many (1:M), registered user to resources
    a. A resource shall be owned/posted by **only one** user.
    b. A user shall be able to post **many** resources.

2. One-to-Many (1:M), registered user to resource comments
    a. A comment shall be posted by **only one** user.
    b. A user shall be able to post **many** comments on a resource.

3. One-to-Many (1:M) registered user to user friend
    a. A friend shall be associated with **only one** user. For example, when user A sends a friend request to user B and the request is accepted, the connection shall be only between those two users.
    b. A user shall be able to accept **many** friend requests and become friends.

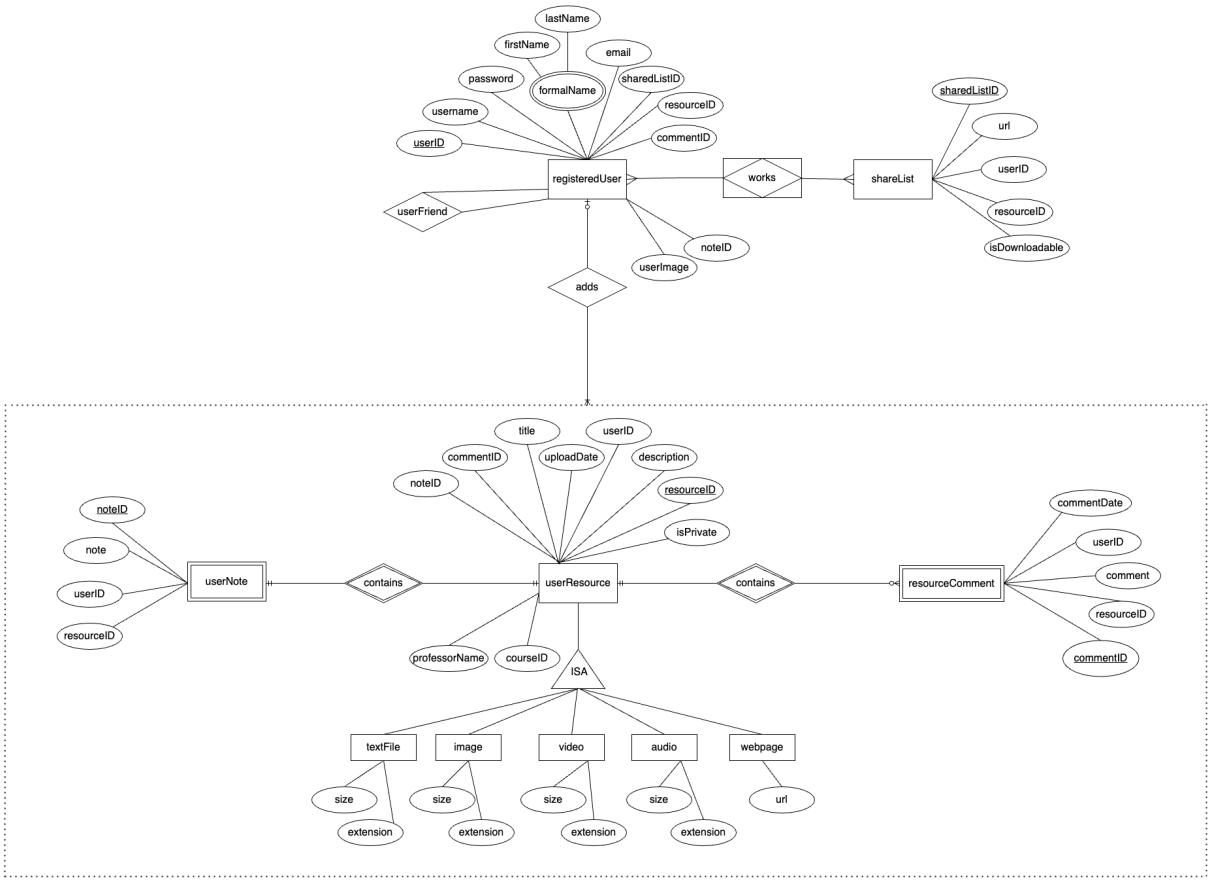
4. Many-to-Many (M:M) registered user to shared list
    a. A registered user shall work on **many** shared lists.
    b. A shared list shall be worked on by **many** registered users.

5. One-to-One (1:1) registered user to note
    a. A note shall be associated with **only one** resource.
    b. A resource shall be able to contain **only one** note.

6. One-to-Many (1:M) resource to resource comments
    a. A comment shall be associated with **only one** resource.
    b. A resource shall be able to contain **many** comments.

7. One-to-One (1:1) resource to user note
    a. A resource shall have **only one** user note.
    b. A user note shall be added to **only one** resource.

Entity Relationship Diagram (ERD):



Database Management System (DBMS):

The DBMS we chose for database management is MySQL because we are familiar with its syntax; in addition, it is simple and efficient.

Media Storage

The multimedia contents will be stored in a mountable file system. The data format for images are jpeg and png, for audio files are mp3 and for video files are mp4.

Search/filter architecture and implementation

The algorithm used to search for registered users in the database starts in the user repository in the searchUsersByUsername method with the SQL statement “SELECT * FROM user WHERE user LIKE :query OR first_name LIKE :query OR last_name LIKE :query”. This statement, which will select entities with identical values in the user, first_name, or last_name attributes, is recognized as a query due to Spring Boot’s Query annotation and then stored into a string parameter called query. The Query is then modified to handle white spaces and then stored into another string variable called newQuery, which is then passed into searchUsersByUsername and returned as an User entry. After that, both the password and email of said user are encrypted via for loop and the entry is then returned if it exists and will return null if it is not.

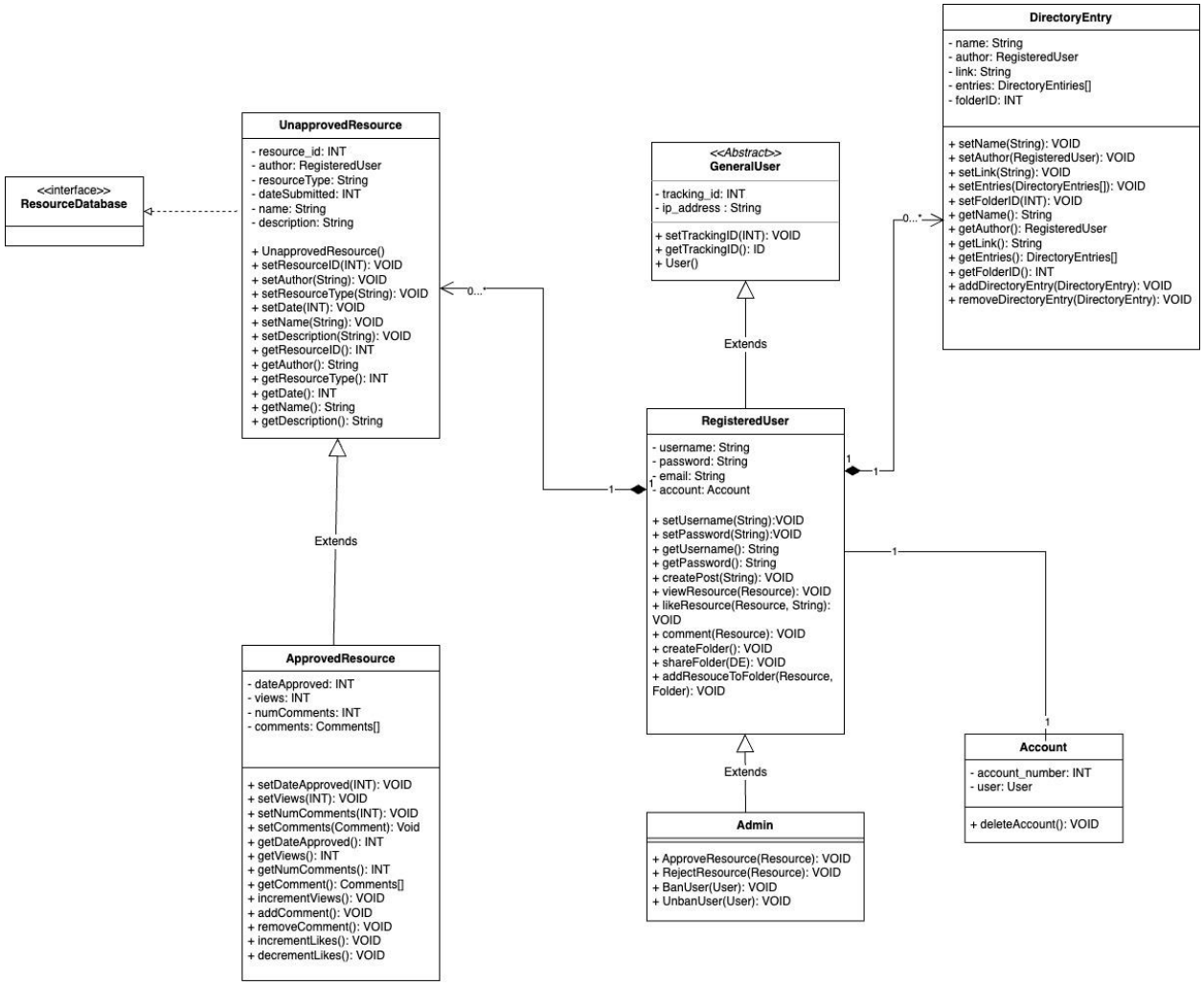
High Level APIs and Main Algorithms

Main Algorithms

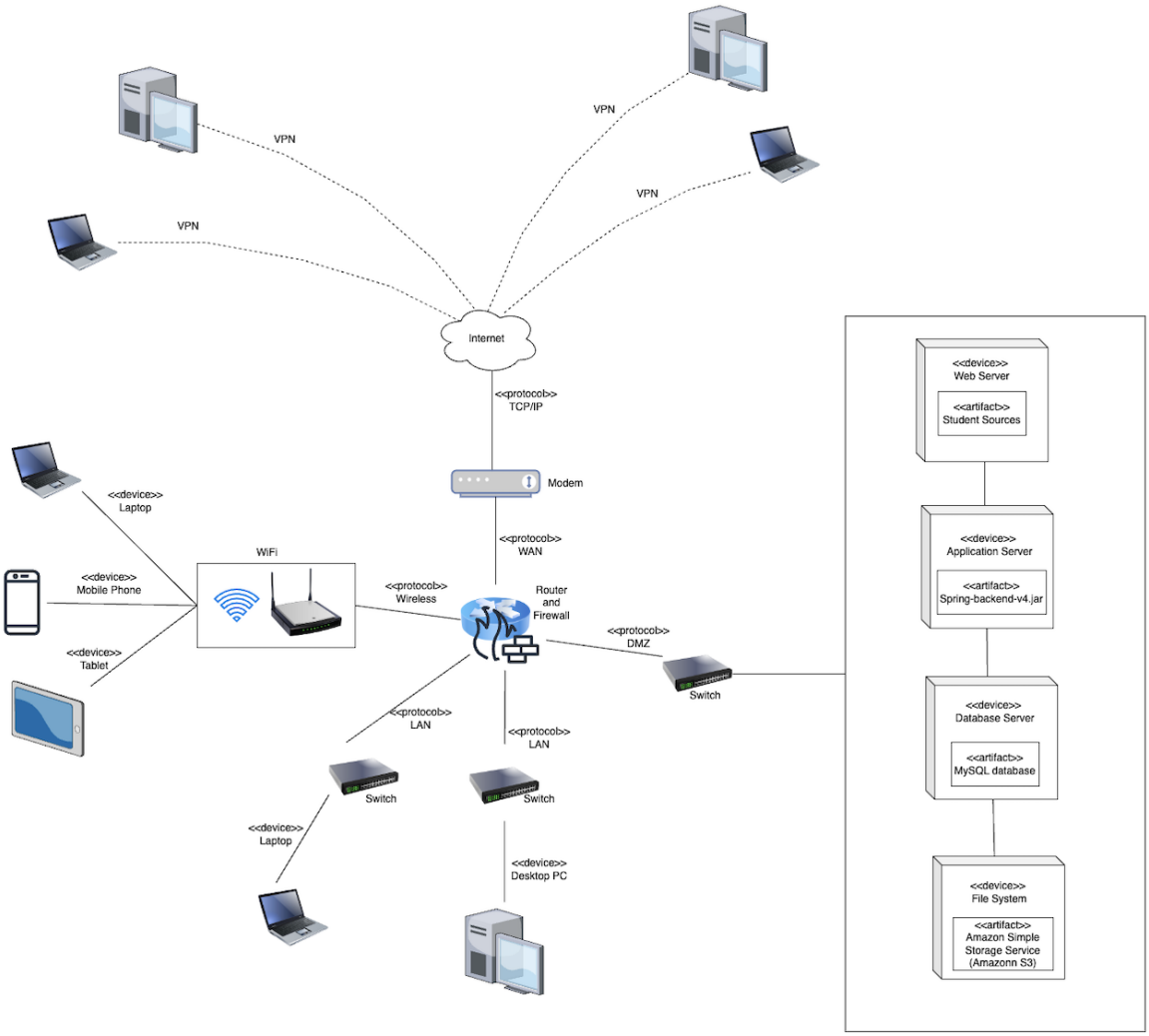
- **Sorting Algorithm:**
  A sorting algorithm will be used for getting the most useful documents first. There are a variety of metrics we can use for sorting such as views, impressions, etc. For example, a user can search through a professor's most viewed documents which will help students get a better understanding of the professor's course. By having a sorting algorithm, we can increase user retention by providing the most relevant documents first.

- **Filter Algorithm:**
  A filtering algorithm allows us to obtain documents of a specific category. Our application should be able to filter through documents by date, course, professor, and/or document type. For example, a user might want only documents that are relevant within the last 2 years or a user might want only documents from a specific professor. By having a filtering algorithm, users can navigate through documents in a convenient manner.

High Level UML Diagrams



High Level Application Network and Development Diagrams



Key Risks in Current Work

Skills Risks

The required skill for the product is knowledge in Java and JavaScript. The skill rating in our team of 7 resulted in an average of 4 on a scale of 1 to 5, with 5 being very familiar and 1 being never used for both Java and JavaScript. This result is a good indication of a member's familiarity with the languages. However, it is understandable that there would be times where members might need help with those languages. In those scenarios, the team would take the time to learn what help the member needs and help the member by providing time and resources.

Schedule Risks

The team is composed of students who are Graduate students and seniors in their Undergraduate Studies. In addition, members are actively involved in extracurricular activities such as school clubs, research, and so on. Apart from these, members dedicate their time and effort in income-generating jobs. At a glance, it can be considered that members have busy schedules and the schedules are bound to conflict. Therefore, to minimize the effect of schedule risk, outside of the regular class time, every member commits an hour to meet on Monday. If members are unable to attend the meeting, they’re updated later in the Discord server by the team lead.

Technical Risks

Technical risk consists of using or referencing code from an unauthorized party. Since the team is a group of students learning to work on a big project, it can be presumed that the internet would be used as a source of reference; hence, there are possibilities where the referenced code might not have permission to be used. To resolve such cases, the team will confirm that the permission to use the code is received before incorporating the code in the project.

Teamwork Risks

In a project that depends heavily on team collaboration, the presence of teamwork risk is inevitable. There are few risks concerning teamwork. First, the decisions might not be unanimous among the members. Second, there could be communication issues such as miscommunication or lack of communication. Lastly, there could be unbalanced workload, meaning some teammates are not able to finish work on time and other members have to work to their maximum capacity or beyond to deliver the task by deadline.

In order to face those obstacles in teamwork, all sides related to a specific decision are explained and pros and cons are weighed; then, the decisions are made accordingly.

Therefore, every member comprehends the effects of the decision. Next, regarding communication issues, the members are encouraged to express their thoughts in team meetings, Discord server. To avoid miscommunication, the members are requested to explain the matter further so that the idea is conveyed clearly. And lastly, regarding unbalanced workload, the team lead ensures that tasks are assigned in a well adjusted manner. However, even though there is an attempt to balance workload, it might not execute perfectly as planned; thus, the efforts of each member are reflected on a contribution score table to make sure the members are credited for their efforts.

Legal/Content Risks

There are two possible risks related to legal/content risks. As a developer, we might encounter difficulties such as copyright issues while using other people/organizations' content. On the other hand, the legal consequences related to terms and conditions designated to end users might arise. To address content risk issues, the team will confirm that the contents being used are not copyrighted. Subsequently, to address legal issues, the terms and conditions document will be descriptive and transparent so that the users find it straightforward and easy to understand.

Project Management

In Milestone 2, we dive deeper into designing our product. The first step for the task management was to divide the Milestone into three parts: Milestone 2 documents, Vertical SW Prototype, and review. To accomplish the goal of completing Milestone on time, internal deadlines were set. The allocated time for Milestone 2 documents was a week; similarly, Vertical SW Prototype was two weeks, and for the review and preparation for submission, it was 2-3 days.

The technologies used for project management were Discord, Zoom, and Trello. The team's main line of communication was a Discord server dedicated to the team. Discord was used to post agendas, reminders, updates on members' progress, feedback, and requests for help. The secondary form of communication was Zoom, where the team met weekly to discuss matters requiring every member's input. The last project management tool used was Trello workspace. On the Trello workspace, different boards were created with checklists and deadlines, which allowed members to view the tasks and status at a glance. In addition, the reminder feature on Trello board was used to remind members of upcoming deadlines.

Milestone 3

Data Definitions V3

General User: A general user is one who doesn't have permission to view any of the approved resources. As they also automatically lack an account on StudentSources, general users do not necessarily need to either log in nor do they have to register and have access to basic features. However, if they choose to do so, general users can sign up by providing an email address, password, and to make sure that they agree to the terms and conditions.

Registered User: Registered users are essentially users who have access to an account on StudentSources. Unlike general users, registered users have the capability to view resources that have been approved as well as have the capability to upload their own. Also, registered users are able to open their own file via creating separate tabs as long as they are logged in and registered into the system.

Account: An account is the one thing that differs between general users and registered users. Accounts store various forms of data belonging to those who are registered users, such as usernames, passwords, full names, and resources that said users have posted. Also, those who have accounts have full access to all approved and unapproved resources and have the capability to search for different files to be added to the list and/or share with other users.

Admin: Admin is a person/ team who is in charge of managing the website. The responsibilities of the admin include maintaining and troubleshooting the website, implementing security protocols, ensuring efficient user experience, and updating the website's contents. Admin also has authoritative power to approve or disapprove resources that users add to the website.

Resources: Resources are the contents added by the users in their account. The list of resources are text files, images/video, online contents. The maximum size of the resources a registered user shall upload would be 1 GB.

- **Text files:** Text files include files with extensions .pdf, and .txt.
- **Images:** Image content includes files with extension .jpg , .jpeg and .png.
- **Video:** Video content will be stored as a link.
- **Audio:** Audio content will be stored as a link.
- **Online Contents:** Online contents include the online resources such as link to webpage, YouTube videos.

Unapproved Resources: Unapproved resources are the resources that are yet to be approved by the admin or the resources that failed to acquire permission from the

admin to be added to the website. The criteria for resources to be approved include the relevancy of the content and if it is related to the course. The unapproved resources are listed as approved resources once they receive approval. Suppose the resources are unsuccessful in gaining permission. In that case, the resource is removed from the website, and the respective user of the resource is informed via the notification panel of the website.

Approved Resources: Approved resources are the resources that are approved based on the approval criteria. Once the resources are approved by the admin, the users can add the content to the list and share it with other users with the generated link.

Directory and Directory Tree: A directory is a folder that contains resources inside. A directory tree is the structure to organize the contents in the directory. Registered users can create multiple directories and subdirectories dedicated to different courses and topics. Registered users can use the default layout or create it for note organization. Registered users can label directories as professor's name, course name, or course abbreviation and number (CSC648, for example). Registered users can share the directories and subdirectories with other users with the generated link.

Comment: Comments are the remarks that users leave for uploaded resources. Only registered users shall be able to leave a comment on a resource. Registered users can leave one or multiple comments on their own resources, shared resources or publicly accessible resources. The posted comments are verified if appropriate by the admin. If comments are not appropriate, the admin can delete the comment from the resource.

Friend: Friend(s) are other registered users who are added as ‘Friend’ by a registered user. Adding other registered users as friends helps find users faster by applying filters while searching. A registered user has options to add/remove friends, and to list friends as ‘Favorite.’

Note: Note(s) are the record of brief descriptions and/or thoughts registered users can add to their accounts. A panel is available for registered users to add notes to their accounts. The associated note panel contains features where a registered user can type and save notes on the website.

Functional Requirements V3

Priority 1

General User

1.1. A general user shall be able to create one or more accounts.
1.2. A general user shall be able to sign up with a unique email address.
1.3. A general user shall be able to sign up with a non-unique password.
1.4. A general user shall be able to see if all requirements for creating a password are met.
1.5. A general user shall be able to sign up with a non-unique username.
1.6. A general user shall be able to read the terms and conditions.
1.7. A general user shall be able to agree to the terms and conditions.

Registered User

2.1. A registered user shall have at least one account.
2.3. A registered user shall be able to view their account information in the Account page.
2.4. A registered user shall be able to temporarily unhide account password information in the *Account* page. By default, when a registered user visits the *Account* page, the password would be hidden for security and password.
2.5. A registered user shall be able to change their account username.
2.6. A registered user shall be able to change their account email.
2.7. A registered user shall be able to change their account password.
2.8. A registered user shall be able to upload any supported file type to their account's cloud storage.
2.10. A registered user shall be able to navigate through files with a directory tree.
2.11. A registered user shall be able to organize files alphabetically through a directory tree.
2.12. A registered user shall be able to view the current directory from a *Files* panel.
2.13. A registered user shall be able to create directories.
2.14. A registered user shall be able to create text files.
2.15. A registered user shall be able to create bookmarks of web URLs.
2.16. A registered user shall be able to pin the resources for easy access in the future.
2.17. A registered user shall be able to view supported files on the website.
2.18. A registered user can view the contents of their files on a *Display* panel.
2.19. A registered user can view the contents of their saved web pages on a *Display* panel.

2.21. A registered user shall have a place to type and save a note for each resource.
2.22. A registered user may have an assistant AI chatbot.
2.24. A registered user shall be able to comment on files.
2.25. A registered user shall be able to delete/move resources to a trash bin.
2.26. A registered user shall be able to delete resources from a trash bin permanently.
2.27. A registered user shall be able to recover items from a trash bin.
2.28. A registered user shall be able to download files onto their computer.
2.30. A registered user shall be able to share their directories by configuring the list as ‘public’.
2.31. A registered user shall be able to configure permissions of their shared directories/folders. The permission allows other users to either only view the list or download the list.
2.33. A registered user shall be able to view other directories shared publicly.
2.35. A registered user shall be able to download other shared directories onto their computer.
2.36. A registered user shall be able to search public directories.
2.40. A registered user shall be able to add other registered users as friends.
2.41. A registered user shall be able to remove other registered users from friend-list.
2.42. A registered user shall be able to mark other registered users as their favorite friend.
2.45. A registered user shall be able to configure visibility of their resources at any time. The visibility can be either public or private.
2.46. A registered user shall be able to message another registered user.
2.47. A registered user shall be able to create a group chat to message multiple other registered users.
2.48. A registered user shall be able to search other registered users whose profile is set public.
2.49. A registered user shall be able to search friends.
2.50. A registered user shall be able to search pinned resources.
2.51. A registered user shall be able to clear notes added to the resource.
2.52. A registered user shall be able to clear comments added to the resource.
2.55. A registered user shall be able to rename their resources.

<u>Account</u>

3.1. Account shall have a certain amount of cloud storage.
3.2. Accounts shall require and store a username that is at least 6 characters.

3.3. Accounts shall require and store a real name tied to them, and the real name must have a first and last name.
3.4. Accounts shall require and store a password tied to them.
3.5. Accounts password shall be encrypted when stored in a database.
3.6. Accounts shall be created by one or more general users.
3.7. An account shall be owned by one registered user.

Admin

4.1. Admin shall be able to access all resources, approved or not
4.2. Admin shall be able to delete any resources present on the website
4.5. Admin shall be able to delete/ban any user for any reason (breaking of rules/TOS, inappropriate username)
4.6. Admin shall be able to delete any inappropriate resources uploaded.
4.8. Admin shall check if comments on resources are appropriate.
4.9. Admin shall delete zero or many inappropriate comments on one or many resources.

Resources

5.1. Resources shall be uploaded by a registered user.
5.2. Resources shall be organized in an alphabetical order.
5.3. Resources such as text files shall be able to be created by a registered user.
5.4. Resources shall be viewed by a registered user.
5.5. Resources shall be commented by registered users.
5.6. Resources shall be moved by a registered user to a trash bin.
5.7. Resources shall be able to be deleted permanently by a registered user from a trash bin.
5.8. Resources shall be recovered by a registered user from a trash bin.
5.9. Resources shall be downloaded on the computer by a registered user.
5.10. Resources shall have comments from other registered users.
5.11. Resources shall be able to be accessed by admin.
5.12. Resources shall be able to be deleted by admin.
5.13. Resources shall be deleted if deemed inappropriate by the admin.
5.15. Resources shall have comments verified by admin.
5.16. Resources shall be pinned by a registered user.
5.17. A resource shall have a note created by a registered user.
5.18. Resources shall have configured visibility by a registered user.
5.19. A pinned resource shall be searched by registered users.
5.20. Resources shall be able to be renamed by registered users who added them.

Directory and Directory Tree

8.1. Directory trees shall be navigated by a registered user.
8.2. Directory trees shall have files organized in alphabetical order.
8.3. Directories shall be created by a registered user.
8.5. Directories shall be able to be shared by registered users by configuring them to be 'public.'
8.6. Directories shall have configured permissions added by a registered user.
8.8 Directories shared by configuring it to 'public' shall be able to be viewed by a registered user.
8.10. Directories shall be able to be downloaded by other registered users onto their own computer.
8.11. Directories of other users registered configured as public shall be able to be searched by registered users.

Comment

9.1. Comments shall be added on resources by registered users.
9.2. Comments on resources shall be verified by admin.
9.3. Comments shall be deleted by the admin, if deemed inappropriate.

Friend

10.1. A friend shall be added by registered users.
10.2. A friend shall be removed by registered users.
10.3. A friend shall be added as a favorite by registered users.
10.4. A friend shall be searched by registered users.

Note

11.1 A note shall be typed and saved by a registered user for each resource.
11.3. Notes shall be cleared by registered users at any time.

Priority 2

General User

1.8. A general user shall be able to resend an account verification email.

Registered User

2.29. A registered user shall be able to share their directories via generated link.
2.32. A registered user shall be able to view other directories shared via link.

2.34. A registered user shall be able to download other shared directories into their account.
2.37. A registered user shall be banned for uploading one or many inappropriate resources by admin.
2.38. A registered user shall not be able to view unapproved resources.
2.39. A registered user shall not be able to share unapproved resources.
2.53. A registered user shall be able to disable comments in the resource.
2.54. A registered user shall be able to delete a group chat they created.

Admin

4.3. Admin shall be able to change an unapproved resource to an approved resource.
4.4. Admin shall be able to change approved resources to unapproved resources.
4.7. Admin shall be able to view unapproved resources.

Unapproved Resources

6.1. An unapproved resource shall only be viewed by an admin.
6.2. An unapproved resource shall not be viewable by users until approved.
6.3. An unapproved resource shall be able to be converted to an approved resource by admin.
6.4. Unapproved resources shall not be able to be shared until an admin approves it.

Approved Resources

7.1. Approved resources shall be able to be viewed by admins and users.
7.2. Approved resources shall be able to be converted back to unapproved resources by the admin.
7.3. Approved resources shall be able to be shared via a generated link.

Directory and Directory Tree

8.4. Directories shall be able to be shared by registered users to other registered users via generated link.
8.7. Directories shared via generated links shall be able to be viewed by a registered user.
8.9. Directories shall be able to be downloaded by other registered users onto their own account.

Comment

9.5. Comments shall be able to be cleared by a registered user at any time.

9.6. Comments shall be disabled by a registered user at any time.

Priority 3

General User

1.9. A general user shall be provided a user manual covering basic features.

Registered User

2.2. A registered user shall be able to interact with a skippable tutorial sequence.
2.9. A registered user shall be able to cancel an upload that is in progress.
2.20. A registered user shall be able to change the color theme of the website.
2.23. A registered user shall be able to highlight text on pdf files.
2.43. A registered user shall be able to edit their posted comments at any time.
2.44. A registered user shall be able to edit their saved notes at any time.

Resources

5.14. Resources such as pdf files shall be highlighted by registered users.

Comment

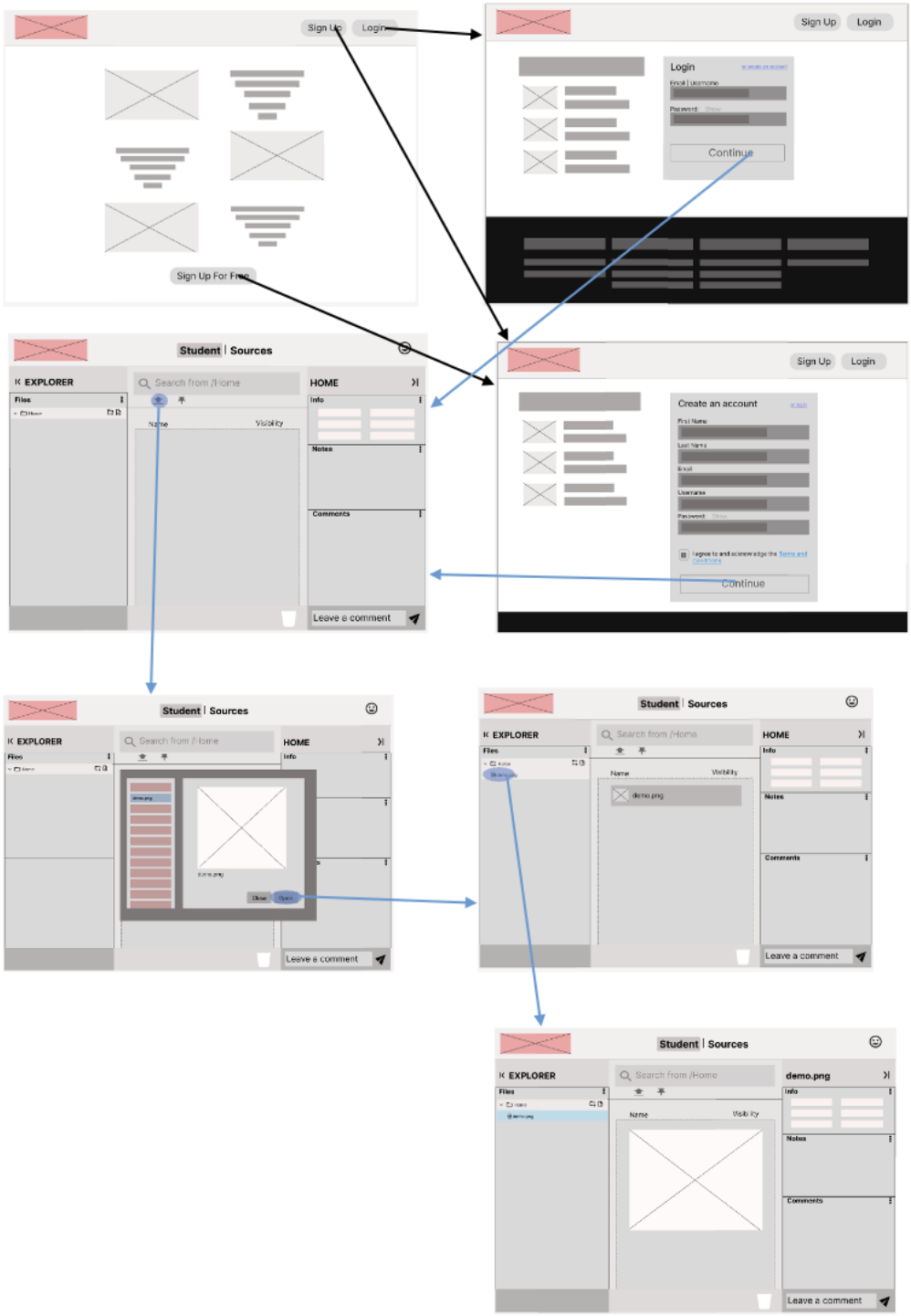
9.4. Comments shall be able to be edited at any time by the registered user who posted it.

Note

11.2. A note shall be able to be edited at any time by the registered user who saved it.

Wireframes V2

To Upload and Open A File



When a user visits the StudentSources site, they land on our homepage where a short description about the product is given. The user can locate the *Login* button on the top-left corner of the home page or the *Sign Up* button on the top-left corner and bottom of the page, just above the footer. *Login* button directs the user to the login page where the user enters email address or username, along with the password. *The Sign Up* button directs the user to the registration page, where the user enters information such as first and last name, email address, username, and password. The user shall also agree to *Terms and Conditions* to continue the registration process. After successfully logging in or signing in, the user is directed to the *Student* page, where the user can upload, make notes, comment and so on. The page is divided into three sections, left, middle and right sections. The left section has two panels. In the top panel of the left section, the *Files* panel, users can see the directory of the selected folder from where the user can select a file to be displayed/previewed in the middle section. The middle section also works as a platform to view all the folders the user has, configure visibility, upload folders/files. The right section has three panels: *Info, Notes, and Comments.* The *Info* panel displays general information about the selected file/folder. The user can use the Notes panel to add their notes for the selected resource. The user can leave one or many comments on the selected resource with the use of the Comments panel.

To upload a file, the user can either drag and drop a file or click on the *Upload* button present in the middle Section. Upon clicking the *Upload* button, the user shall be prompted to choose file(s) from their local device and the selected file(s) shall be uploaded to their account. To view the contents of a file, the user can select the file on the *Files* panel, then the file will be opened in the middle section.

To Message Other User



From the previous *Open a File* flow, to exit the file open screen, the user can click on the root directory (*Home*, in this case). The user can find the Inbox feature by navigating to the Nav-Bar and clicking on the *User* icon. After clicking the *Inbox* option, a new window shall be opened where users can create group chats by clicking on the Create *Group Chat* button. The user also has the option to have one-to-one conversation by clicking on the Add icon on the Create *DM* section. To make it easier for users, users can search for other users in order to message them or add to a group chat.

To Search Other User's resources



To search for other users or their resources, the user can click on the *Sources* page located at the top of the current page. The user will be directed to the Search page where they can search with different filter options such as *Users, Folders, Bookmarked or Friends.* Once the search is complete and the results are outputted, the user can select the user they wish to see the resources of.

High Level Database Architecture and Organization V2

This high level conceptual database design will be based on one of the unique features StudentSources provides, which is the exclusivity to SFSU students and being able to have a dedicated community for SFSU study resources.

Database Organization

Database Requirements:

1. A registered user shall be able to upload zero or many resources.
2. A registered user shall add zero or many comments on their own resources.
3. A registered user shall comment on many other registered users' resources.
4. A registered user shall add zero or many other registered users as friends.
5. A registered user shall remove zero or many other registered users from friend-list.
6. A registered user shall add zero or one note for each resource.
7. A registered user shall set permissions to ‘downloadable’ of many of their shared lists.
8. A registered user shall set permissions to ‘view only’ of many of their shared lists.
9. A registered user shall set visibility as ‘public’ of many of their resources.
10. A registered user shall set visibility as ‘private’ of many of their resources.

Entities:

1. registeredUser
2. userResource
3. resourceComment
4. userFriend
5. sharedList
6. userNote

Entity Attributes and Domains:

1. registeredUser
    a. userID (primary key) = INT, numeric, unique, and not null

    b. username = VARCHAR(32), not null, alphanumeric, can be upper and lowercase, must be unique

c. email = VARCHAR(50), not null, alphanumeric, must contain valid email with a @

d. password = VARCHAR(32), not null, alphanumeric, will be encrypted before storing, must have requirements specified by website

e. formalName = VARCHAR(32), first and last name of user, can only contain letters

f. userImage = BLOB, user profile image saved as binary data

g. sharedListID (foreign key) = INT, numeric, references sharedList's sharedListID

h. resourceID (foreign key) = INT, numeric, references userResources' resourceID

i. commentID (foreign key) = INT, numeric, references resourceComment's commentID

j. noteID (foreign key) = INT, numeric, references userNote's noteID

2. userResource
    a. resourceID (primary key) = INT, numeric, unique, not null, auto incremental

    b. title = VARCHAR(32), not null, alphanumeric, must be unique to prevent confusion of resources.

    c. description = TEXT, can contain anything TEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 65,535 bytes.

    d. uploadDate = DATE, Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'

    e. courseID = VARCHAR(12), alphanumeric, will be stored as 3-4 letters following 3 numbers, for example: CSC101

f. professorName = VARCHAR(32), alphanumeric, can only contain letters and hyphens

g. isPrivate = TINYINT(1), equivalent to boolean data type, can only contain 0 or 1, tracks if the resource is private or public

h. userID (foreign key) = INT, numeric, references registeredUser's userID

i. commentID (foreign key) = INT, numeric, references resourceComment's commentID

j. noteID (foreign key) = INT, numeric, references userNote's noteID

userResource entity contains child entities which define the format of resources that users store. The child entities with their respective attributes are as follows:

i. textFile
   a. size = INT, numeric
   b. extension = VARCHAR(10), alphanumeric, can contain letters and a special character like '.'

ii. image
   a. size = INT, numeric
   b. extension = VARCHAR(10), alphanumeric, can contain letters and a special character like '.'

iii. video
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

iv. audio
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

v. webpage
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

3. resourceComment
   a. commentID (primary key) = INT, numeric, not null, unique

   b. comment = TINYTEXT, alphanumeric, can contain anything TINYTEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 255 characters.

   c. commentDate = DATE, Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'

   d. resourceID (foreign key) = INT, numeric, references userResource's resourceID field, creating a relationship between the resource and the comment that was posted on the resource.

   e. userID (foreign key) = INT, numeric, references registeredUser's userID field, creating relationship between the comment and the user who commented it.

4. userFriend
   a. UID1 (primary key and foreign key) = INT, numeric, not null, references registeredUser's userID
   b. UID2 (primary key and foreign key) = INT, numeric, not null, references registeredUser's userID

5. sharedList
   a. sharedListID (primary key) = INT, numeric, not null

   b. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

   c. isDownloadable = TINYINT(1), equivalent to boolean data type, can only contain 0 or 1, tracks if the resource is view only or is downloadable by other registered users

   d. userID (foreign key) = INT, numeric, references registeredUser's userID

   e. resourceID (foreign key) = INT, numeric, references userResource's resourceID

6. userNote
   a. noteID (primary key) = INT, numeric, not null,

b. note = TINYTEXT, alphanumeric, can contain anything TINYTEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 255 characters.

c. userID (foreign key) = INT, numeric, references registeredUser's userID

d. resourceID (foreign key) = INT, numeric, references userResources' resourceID

Entity Relationships:

1. One-to-Many (1:M), registered user to resources
    a. A resource shall be owned/posted by **only one** user.
    b. A user shall be able to post **many** resources.

2. One-to-Many (1:M), registered user to resource comments
    a. A comment shall be posted by **only one** user.
    b. A user shall be able to post **many** comments on a resource.

3. One-to-Many (1:M) registered user to user friend
    a. A friend shall be associated with **only one** user. For example, when user A sends a friend request to user B and the request is accepted, the connection shall be only between those two users.
    b. A user shall be able to accept **many** friend requests and become friends.

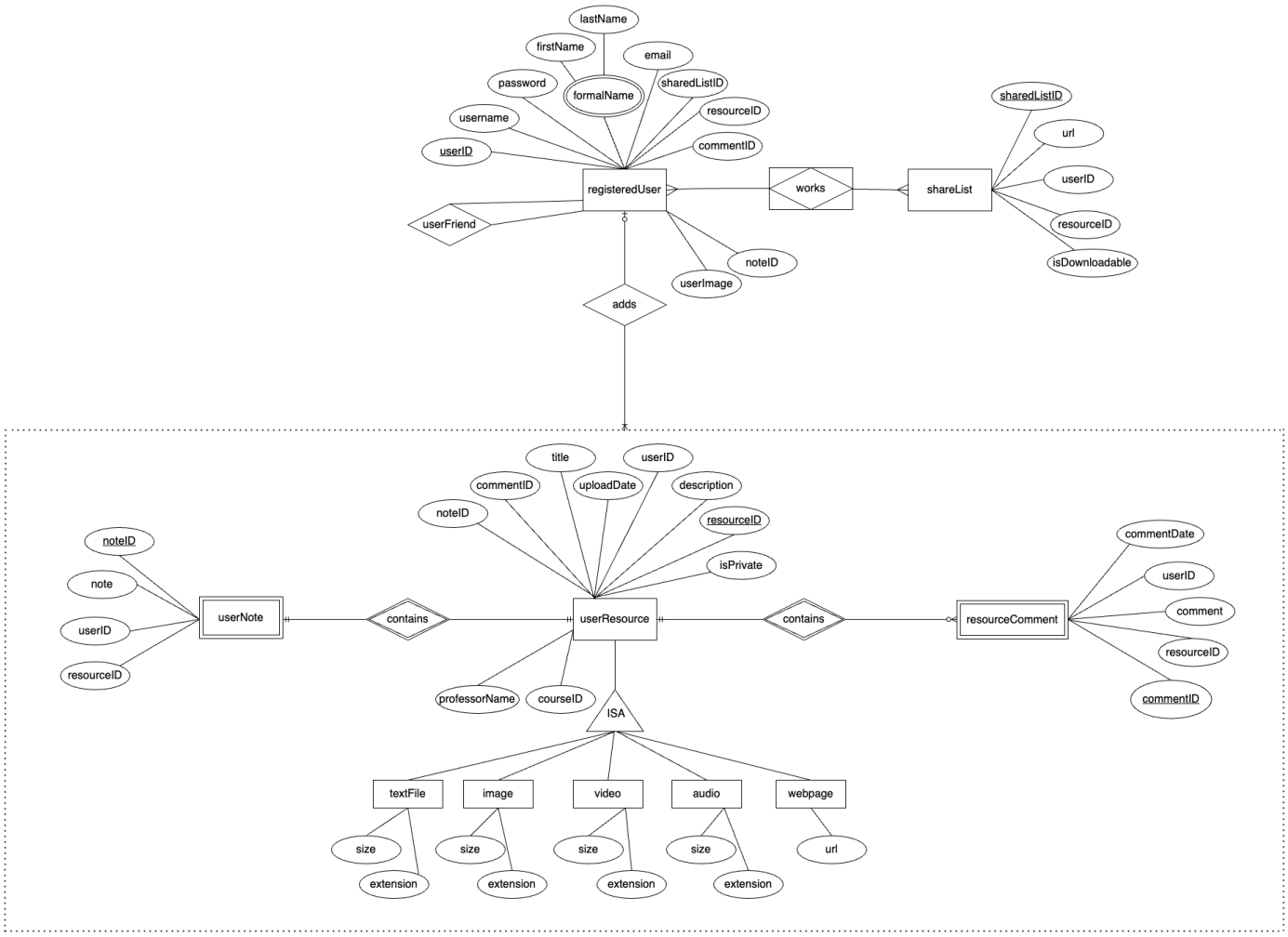
4. Many-to-Many (M:M) registered user to shared list
    a. A registered user shall work on **many** shared lists.
    b. A shared list shall be worked on by **many** registered users.

5. One-to-One (1:1) registered user to note
    a. A note shall be associated with **only one** resource.
    b. A resource shall be able to contain **only one** note.

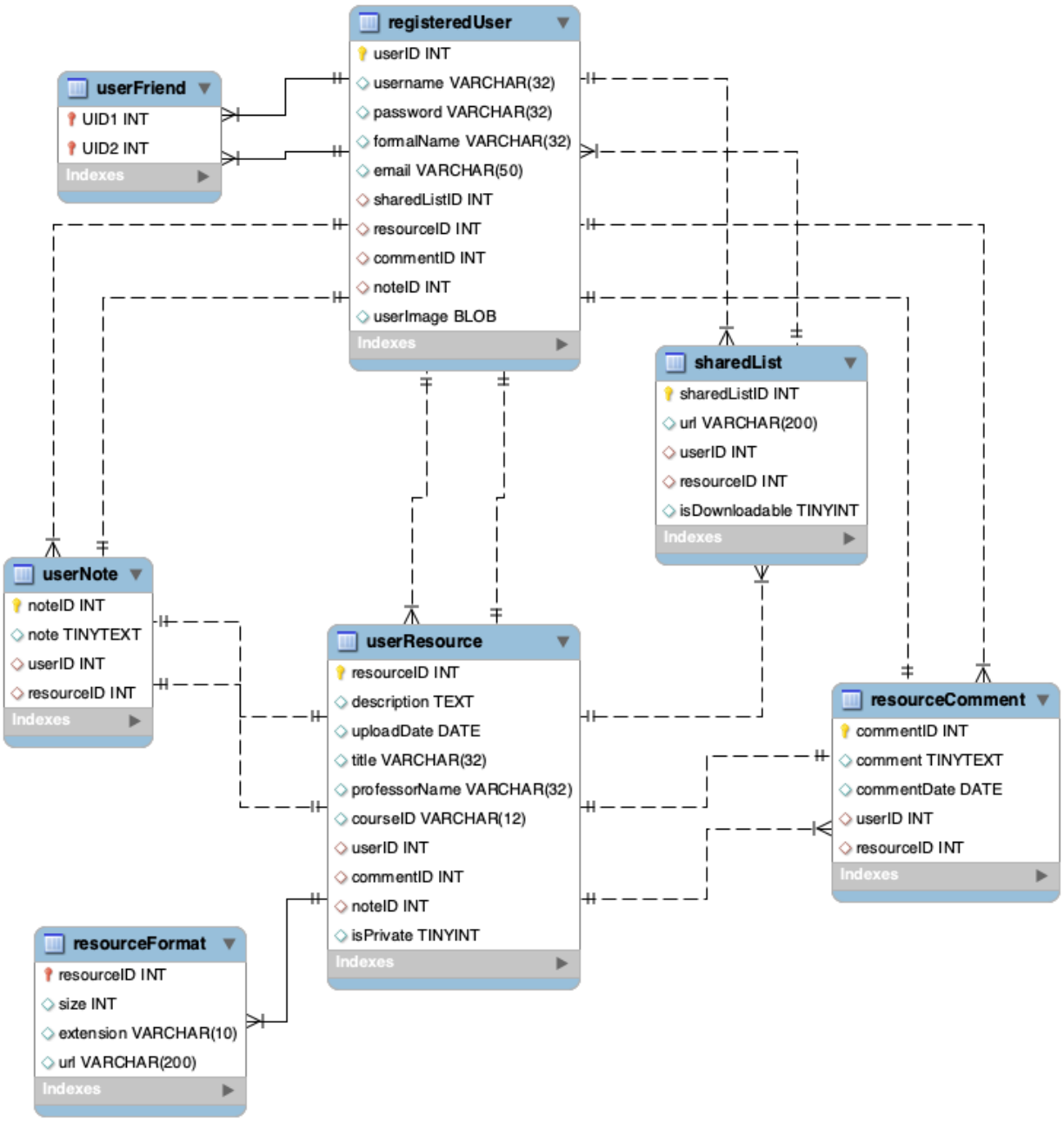
6. One-to-Many (1:M) resource to resource comments
    a. A comment shall be associated with **only one** resource.
    b. A resource shall be able to contain **many** comments.

7. One-to-One (1:1) resource to user note
    a. A resource shall have **only one** user note.
    b. A user note shall be added to **only one** resource.

Entity Relationship Diagram (ERD)



Enhanced Entity Relationship (EER) Diagram



Database Management System (DBMS)

The DBMS we chose for database management is MySQL because we are familiar with its syntax; in addition, it is simple and efficient.

Media Storage

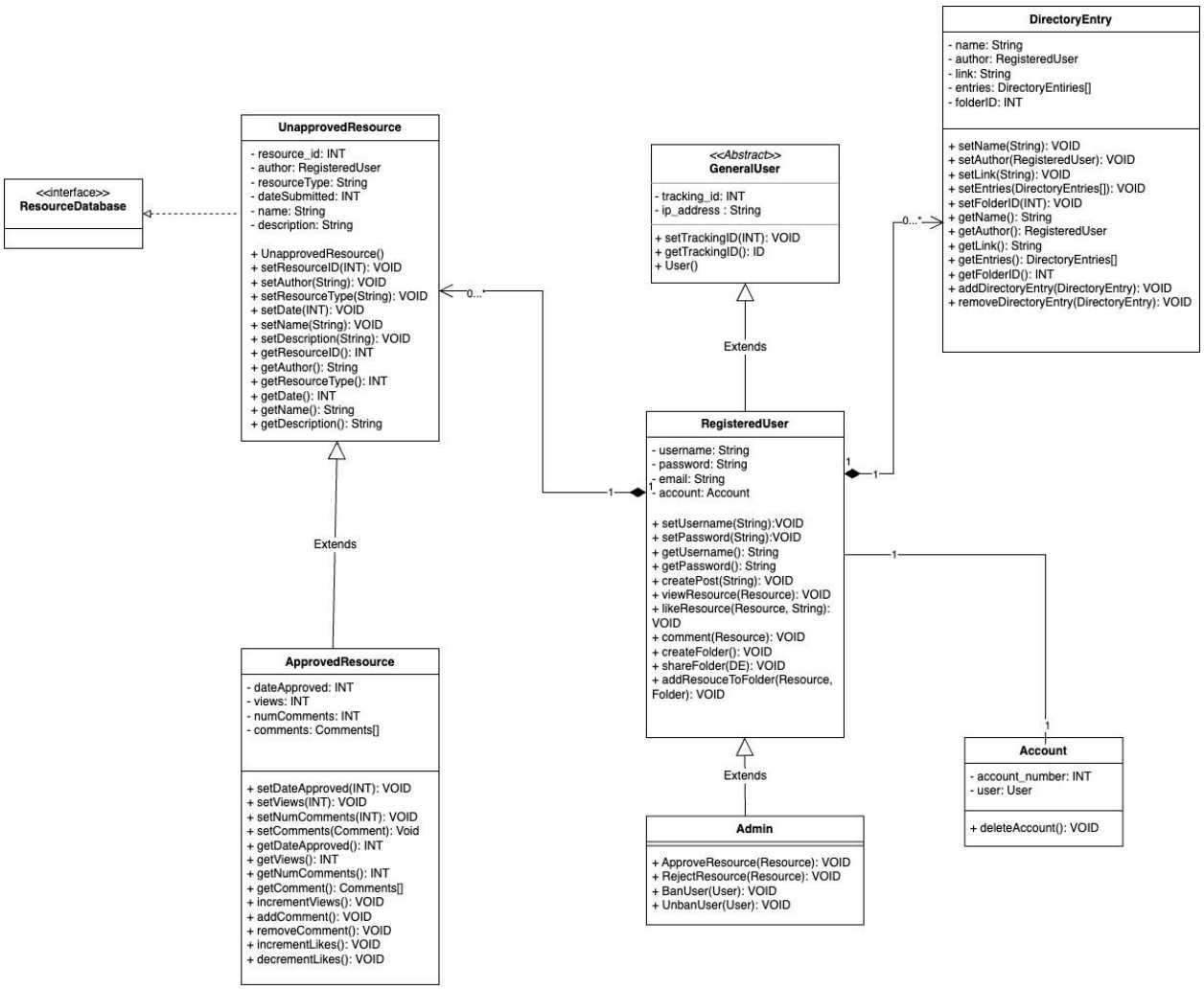
The multimedia contents will be stored in a mountable file system. The data format for images are jpeg and png, for audio files are mp3 and for video files are mp4.

Search/filter architecture and implementation

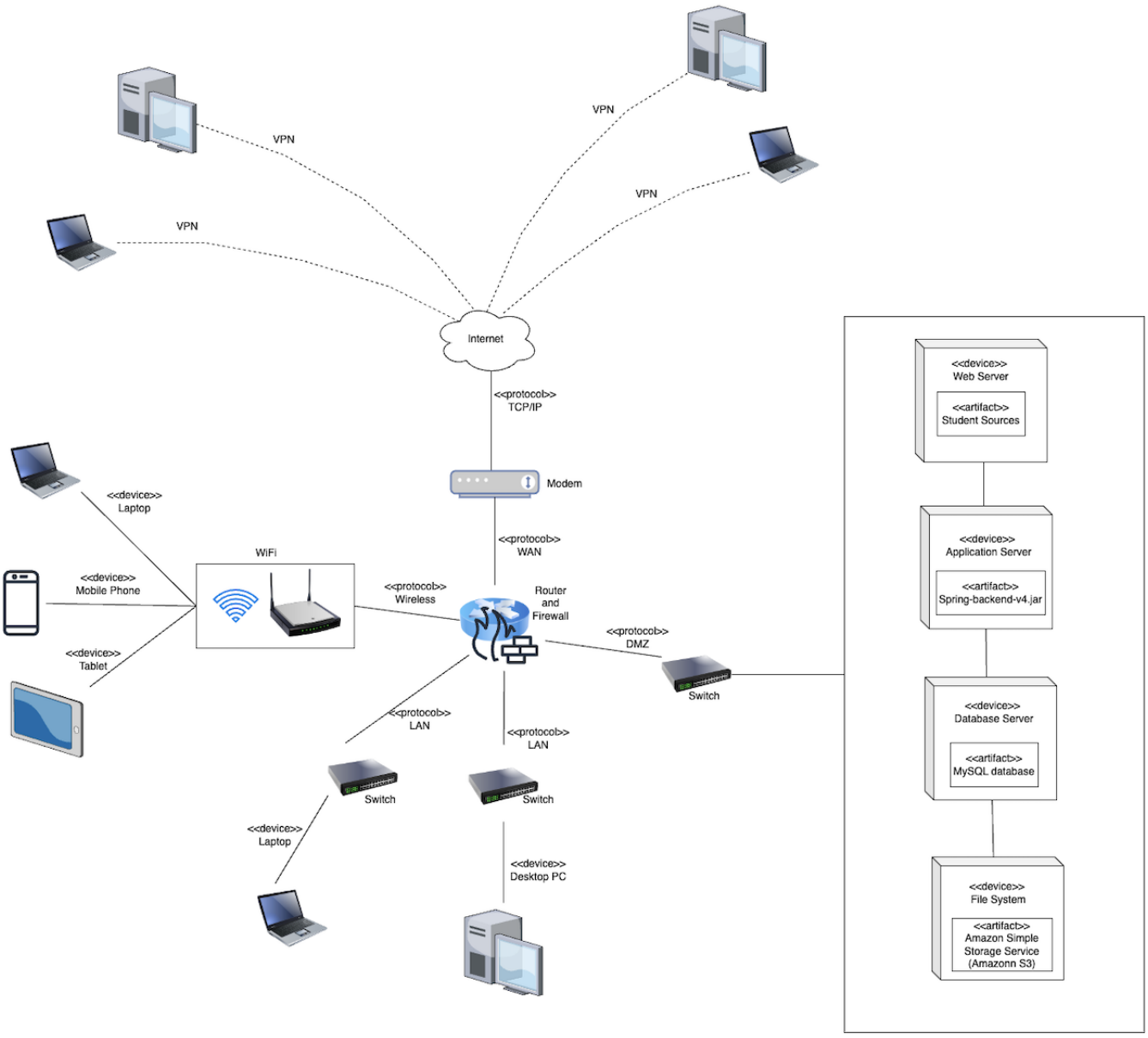
The algorithm used to search for registered users in the database starts in the user repository in the searchUsersByUsername method with the SQL statement “SELECT * FROM user WHERE user LIKE :query OR first_name LIKE :query OR last_name LIKE :query”. This statement, which will select entities with identical values in the user, first_name, or last_name attributes, is recognized as a query due to Spring Boot’s Query annotation and then stored into a string parameter called query. The Query is then modified to handle white spaces and then stored into another string variable called newQuery, which is then passed into searchUsersByUsername and returned as an User entry. After that, both the password and email of said user are encrypted via for loop and the entry is then returned if it exists and will return null if it is not.

High Level Diagrams V2

UML Diagram



Deployment and Network Diagram



Milestone 4

Product Summary

Project Name: StudentSources

Committed Functions

1. A general user shall be able to create one or more accounts.
2. A general user shall be able to sign up with a unique email address.
3. A general user shall be able to sign up with a non-unique password.
4. A general user shall be able to see if all requirements for creating a password are met.
5. A general user shall be able to sign up with a non-unique username.
6. A general user shall be able to read the terms and conditions.
7. A general user shall be able to agree to the terms and conditions.
8. A registered user shall have at least one account.
9. A registered user shall be able to view their account information in the Account page.
10. A registered user shall be able to temporarily unhide account password information in the *Account* page. By default, when a registered user visits the *Account* page, the password would be hidden for security and password.
11. A registered user shall be able to change their account username.
12. A registered user shall be able to change their account email.
13. A registered user shall be able to change their account password.
14. A registered user shall be able to upload any supported file type to their account's cloud storage.
15. A registered user shall be able to navigate through files with a directory tree.
16. A registered user shall be able to organize files alphabetically through a directory tree.
17. A registered user shall be able to view the current directory from a *Files* panel.
18. A registered user shall be able to create directories.
19. A registered user shall be able to create text files.
20. A registered user shall be able to create bookmarks of web URLs.
21. A registered user shall be able to pin the resources for easy access in the future.
22. A registered user shall be able to view supported files on the website.
23. A registered user can view the contents of their files on a *Display* panel.

24. A registered user can view the contents of their saved web pages on a *Display* panel.
25. A registered user shall have a place to type and save a note for each resource.
26. A registered user shall be able to comment on files.
27. A registered user shall be able to delete/move resources to a trash bin.
28. A registered user shall be able to delete resources from a trash bin permanently.
29. A registered user shall be able to recover items from a trash bin.
30. A registered user shall be able to download files onto their computer.
31. A registered user shall be able to configure permissions of their shared directories/folders. The permission allows other users to either only view the list or download the list.
32. A registered user shall be able to share their directories by configuring the list as ‘public’.
33. A registered user shall be able to view other directories shared publicly.
34. A registered user shall be able to download other shared directories onto their computer.
35. A registered user shall be able to search public directories.
36. A registered user shall be able to add other registered users as friends.
37. A registered user shall be able to remove other registered users from friend-list.
38. A registered user shall be able to mark other registered users as their favorite friend.
39. A registered user shall be able to configure visibility of their resources at any time. The visibility can be either public or private.
40. A registered user shall be able to message another registered user.
41. A registered user shall be able to create a group chat to message multiple other registered users.
42. Account shall have a certain amount of cloud storage.
43. Accounts shall require and store a username that is at least 6 characters.
44. Accounts shall require and store a real name tied to them, and the real name must have a first and last name.
45. Accounts shall require and store a password tied to them.
46. Accounts password shall be encrypted when stored in a database.
47. Accounts shall be created by one or more general users.
48. An account shall be owned by one registered user.
49. Resources shall be uploaded by a registered user.
50. Resources shall be organized in an alphabetical order.

51. Resources such as text files shall be able to be created by a registered user.
52. Resources shall be viewed by a registered user.
53. Resources shall be commented by registered users.
54. Resources shall be moved by a registered user to a trash bin.
55. Resources shall be able to be deleted permanently by a registered user from a trash bin.
56. Resources shall be recovered by a registered user from a trash bin.
57. Resources shall be downloaded on the computer by a registered user.
58. Resources shall have comments from other registered users.
59. Resources shall be pinned by a registered user.
60. A resource shall have a note created by a registered user.
61. Resources shall have configured visibility by a registered user.
62. Directory trees shall be navigated by a registered user.
63. Directory trees shall have files organized in alphabetical order.
64. Directories shall be created by a registered user.
65. Directories shall have configured permissions added by a registered user.
66. Shared directories shall be able to be viewed by a registered user.
67. Directories shall be able to be downloaded by other registered users onto their own computer.
68. Directories of other registered users shall be able to be searched by registered users.
69. Comments shall be added on resources by registered users.
70. A friend shall be added by registered users.
71. A friend shall be removed by registered users.
72. A friend shall be added as a favorite by registered users.
73. A note shall be typed and saved by a registered user for each resource.
74. A registered user shall be able to search other registered users whose profile is set public.
75. A registered user shall be able to search friends.
76. A registered user shall be able to search pinned resources.
77. A registered user shall be able to clear notes added to the resource.
78. A registered user shall be able to clear comments added to the resource.
79. A pinned resource shall be searched by registered users.
80. Comments shall be able to be cleared by a registered user at any time.
81. A friend shall be searched by registered users.
82. Notes shall be cleared by registered users at any time.

Unique Features

We have many unique features that make our website, StudentSources, stand out from the rest. The unique features are as follows: a built-in AI chatbot, a group chat feature, and other ways for students to collaborate such as sharable resources and direct messaging. StudentSources' built-in chatbot is a hot new feature that many websites are adding to enhance the user experience and save the user lots of time when searching for an answer. In addition to the AI chatbot, StudentSources promotes collaboration among students. Through the platform, students can easily share resources, exchange ideas, and collaborate on projects. They can comment on each other's work, direct messages to each other, create a group chat as study group, and have the ability to add or remove friends to stay in touch with users with similar classes and resources. These features make our website an ideal tool for students to enhance their academic experiences and store their study resources.

Product URL: http://54.215.216.18/

Usability test plan

Test Objectives Per Function

1.
Function Being Tested:
A registered user shall be able to search other registered users.

Objective
To determine if users are able to use the search bar with ease and apply filters.

2.
Function Being Tested:
A registered user shall be able to upload any supported file type to their account's cloud storage.

Objective
To determine if users can upload files in a few steps.

3.
Function Being Tested:
A registered user shall be able to delete resources to a trash bin.

Objective
To determine if users find this feature useful in a website for notes organization.

4.
Function Being Tested:
A registered user shall be able to pin the resources for easy access in the future.

Objective
To determine if users find this feature to be efficient.

5.
Function Being Tested:
A note shall be typed and saved by a registered user for each resource.

Objective
To determine users' opinion having a space to add notes for each resource.

Test Description

System Setup

Participants were provided with the website URL so that they are able to access it on your local computer. Testing was performed through physical and virtual presence. In the physical testing environment, the moderator was present in the same room as the participant and observed and recorded the process while a user was navigating through the website. In the virtual environment, participants were requested to share their screen while they carry out the different tasks being tested.

The environment set up required to efficiently perform the test included:

1. Availability of computer devices such as Desktop Computer, Laptop, Tablet or Mobile Phones
2. Availability of Operating Systems such as Microsoft Windows, or MacOS
3. Internet connection to the above mentioned devices either via Ethernet or Wireless connection
4. Availability of web browsers such as Google Chrome, Firefox, Edge, or Safari

Starting Point

The starting point for the test was users were provided with a brief explanation of the website and tasks description. Once users were ready to perform the test, they started from the landing page of the website. If users wished to create an account, they had freedom to do so, otherwise an existing demo account was provided for users to login to the portal. Once, logged in users carried out the tasks listed in the prompt and a moderator recorded all the details. Users were also requested to fill out tables and survey questionnaires to help developers understand user perspective regarding the usability of the website.

Intended Users

The primary targeted users for *StudentSources* were students. Since an effective organization of study notes assists students succeed in their academic goal, further assisting them into their professional career, the participants of the test were students who are in their undergraduate or graduate level studies. While the product is intended for students of any level, we believe that students pursuing higher education would benefit the most from our website. All users participated in the test voluntarily.

The steps for the sample users were as follow:

1. Search

    a. Search other users
2. Upload
    a. Upload a file
3. Pin
    a. Pin a file
    b. Unpin a previously pinned file
4. Delete
    a. Delete a file
5. Notes
    a. Add notes to a file
    b. Clear notes

URL

The website can be accessed using following url:
http://54.215.216.18/

Factors to be measured

With usability testing, we planned to measure the effectiveness and efficiency of the website. Along with the performance factors, we also observed and analyzed the users' satisfaction while using the website.

Usability Test Table: Effectiveness

Test 1

Test/Use Case	% Completed	Errors	Comments
Search	100%	No display of messages when user isn't present	Add display message when no user found
Upload	100%	None	Very fast!
Delete	100%	None	File deleted
Pin	100%	None	It'd be better if users can pin folders from the side panel itself.
Notes	100%	None	No comments

Test 2

Test/Use Case	% Completed	Errors	Comments
Search	100	No errors	A little unintuitive for Student vs Source tab for searching for users
Upload	100	No errors	Very intuitive to standard processes, Drag and drop doesn't work for multiple files, Also doesn't specify what files are accepted for upload, i.e. Similar to how ilearn uploads told users what files were accepted. I was thinking if there could be a more aesthetic approach to Name and Visibility separation with the uploaded files, since everything there has a white background and is right next to each other downwards. Even a

			slight padding to give a separation might help in my opinion.
Delete	100	No errors	Worked fine for 3 vertical dot drop down
Pin	100	No errors	When website is minimized I mistook the wrong pin as pinning the uploaded,
Notes	50	No errors	Clear Notes wasn't working. When adding Notes, for instance clicking on the Text file the middle screen feels as if it's the Notetaking area rather than the right side where it acts not only as the miscellaneous, but also the actual Notes and Comments section.

Test 3

Test/Use Case	% Completed	Errors	Comments
Search	100%	No errors	The search function worked fine
Upload	100%	No errors	Uploading files worked fine
Delete	100%	No errors	Deleting files worked fine but would like to have some more clarity on when clearing the files in the trash (warning before deleting)
Pin	100%	No errors	Pinning files worked perfectly
Notes	100%	- Clear notes button was not working - Same notes were saved across all files	The clear notes button did not work when pressed. The notes are not separate for each individual file.

Test 4

Test/Use Case	% Completed	Errors	Comments
Search	70%	Not implemented	Need to show result
Upload	100%	No error	ok
Delete	80%	Delete but still showing file	Need to delete completely
Pin	80%	Unable to see link	Need to display pin
Notes	80%	No errors also unable to delete note	Need delete completely if asked

Test 5

Test/Use Case	% Completed	Errors	Comments
Search	100%	in some cases, the search results would show ‘to be implemented’	Really liked how the search results will link directly to the file when clicked on.
Upload	100%	No errors	Very quick, although unclear which files are supported.
Delete	100%	No errors	Trash was easy to understand
Pin	100%	No errors	Was expecting pins to always be displayed at the top instead of toggling pins on/off.
Notes	100%	No errors	Notes worked as expected

Usability Test Table: Efficiency

Test/Use Case	Time		Effort	Content / Design	
	Avg. time to complete (in sec)	Avg. time of users who completed / Avg. time of all users (in sec)	Avg. numbers of steps taken to complete (rounded to nearest whole number)	Avg. Number of screens	Number of pages of instructions
Search	64	1.21	3	1	1
upload	5.2	1	3	1	1/3
Delete	12.6	1.15	3	1	1/3
Pin	7.2	1.08	2	1	1/3
Notes	41	1.50	2	1	1/2

User Satisfaction

1.

Function: Search *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
It was easy to use search bar.	○	○	○	○	○
I can use part of a user's name to search their profile.	○	○	○	○	○
The results from search function were satisfactory.	○	○	○	○	○

2.

Function: Upload *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
It was simple to upload a file.	○	○	○	○	○
Uploads take less than a minute.	○	○	○	○	○
I can upload multiple files.	○	○	○	○	○

3.

Function: Comment *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
I was able to add a comment on my file.	○	○	○	○	○
I was able to disable comment.	○	○	○	○	○
I was able to clear comments with few clicks.	○	○	○	○	○

4.

Function: AI Chatbot *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
The chatbot helped me with my needs.	○	○	○	○	○
The chatbot answered fast.	○	○	○	○	○
I liked the feature of the chatbot in this system.	○	○	○	○	○

5.

Function: Notes *

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

	1	2	3	4	5
I can add notes easily.	○	○	○	○	○
I was able to clear notes.	○	○	○	○	○
Notes can be copied.	○	○	○	○	○

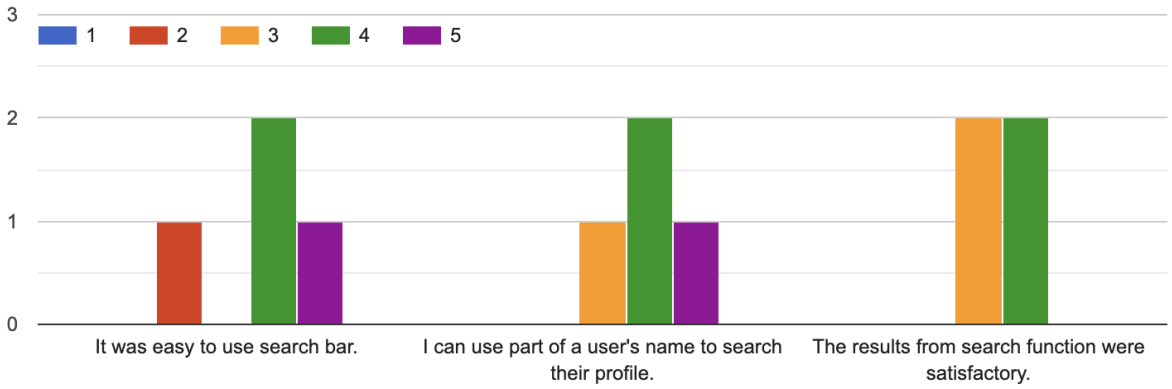
Results of User Satisfaction Evaluation

1.

Function: Search

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

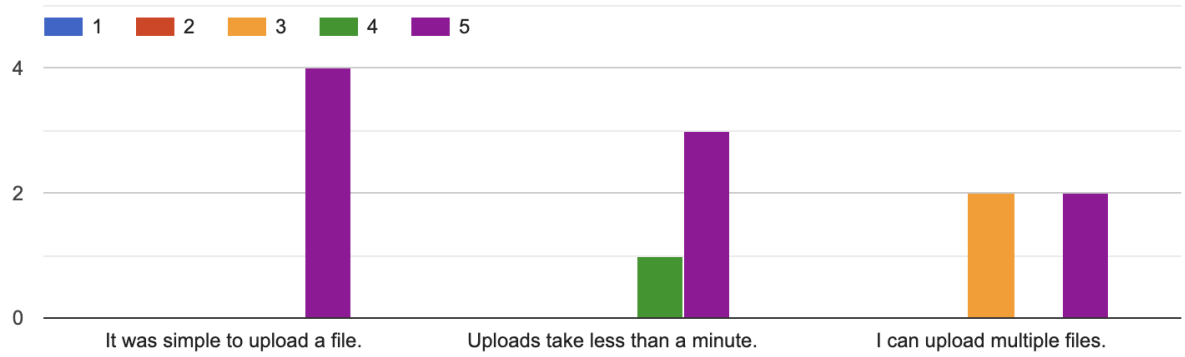


2.

Function: Upload

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

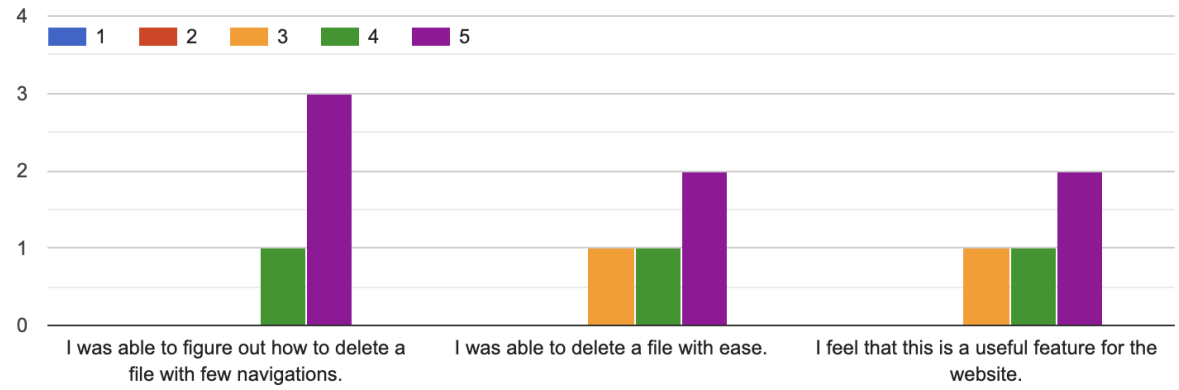


3.

Function: Delete

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

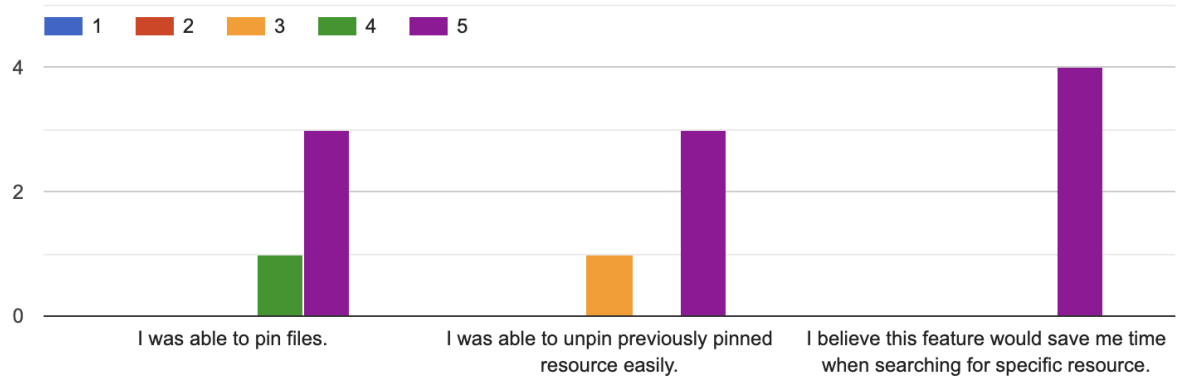


4.

Function: Pin

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)

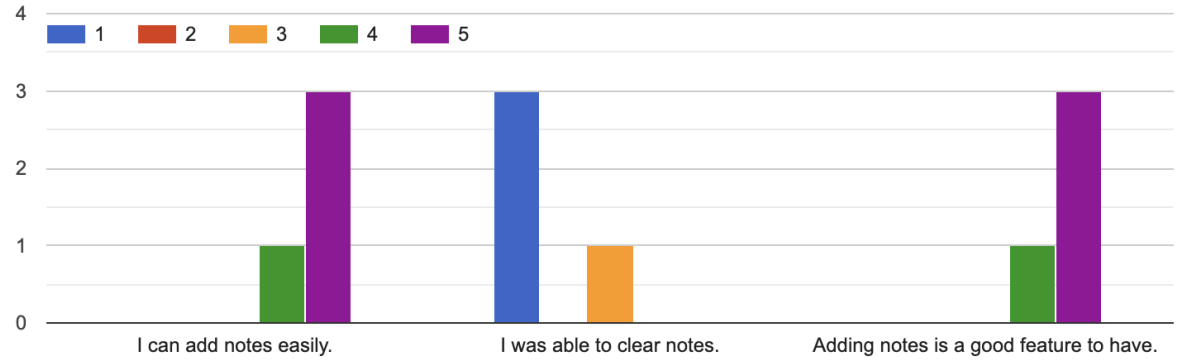


5.

Function: Notes

Copy

Please choose whether you agree or disagree on a scale from 1 (strongly disagree) to 5 (strongly agree)



QA test plan

Test Objectives

The objective of this test is to determine the quality of listed factors:
security - check if the user password is encrypted before being saved to database
storage - ensure that user is aware of the remaining storage capacity
- confirm if the system is able to handle large file sizes
performance - confirm if website is able to handle large number of users simultaneously
compatibility - check is the website is compatible with various web browsers

HW and SW Setup

The hardware setup for this testing were laptops with different operating systems such as Windows, Linux and MacOS. The connection to the wireless WIFI was also secured before the testing began.

The software setup for this test were AWS EC2 instances, Amazon MySQL Relational Database System, Amazon Simple Storage Service. The tests were performed using different web browsers such as Google Chrome, Safari, Mozilla Firefox and Edge.

The software state where testing began were:

1. Password encryption - *Sign up/Login* Page
2. Display storage capacity - *Student* Page
3. Handle large files - *Student* Page
4. Multiple users simultaneously - *Landing* Page
5. Compatible with several web browsers - *Landing* Page

Test Environment

The test environment fulfilled all the requirements of the following checklist:
All required hardware available and connected properly
All required software configured and connected properly
Presence of standard test data sets

In terms of human resources the test required 1 to 2 developers per feature. The expected time to complete testing one feature was less than 1 minute to max 3 minutes.

Risks and Contingencies

1. Late delivery of program code
2. Conflict in the schedule
3. Resources like systems, skills of individuals not properly tracked
4. Failure to resolve responsibilities

Feature to Be Tested

1. System shall always hide registered users account password information on startup the Account page.
2. System shall provide information to registered users the amount of free storage remaining.
3. StudentSources shall be able to handle a large number of simultaneously active users.
4. StudentSources shall be able to handle files with a large size, with the max size being 1 GB.
5. Website shall be compatible with a wide range of web browsers, including but not limited to: Chrome, Firefox, Edge, Safari.

QA Test plan

S.N.	Test Title	Test Description	Test Input	Expected Output	Pass/Fail
1	Security	Check if the user password is being encrypted before storing in the database	Entered password	The password is encrypted	Pass
2	Storage	Ensure that user is aware of the remaining storage capacity	Displayed available storage	The user's remaining storage should be visible	Pass
3	Performance	Check if the website can	Multiple testers used website	The site should function the	Pass

		handle multiple users simultaneously	simultaneously	same with multiple concurrent users	
4	Storage	Check if the system can handle large files	Uploaded 30KB	The file should upload	Pass
5	Compatibility	Check if the supports different web browsers	Site opened in Safari, Google Chrome and Mozilla Firefox	The site should open and work similarly without loss of functionality in different browsers.	Pass

Code Review

Coding Style

Our agreed coding style prioritizes the use of proper naming conventions, specifically camelCase for functions and classes. Additionally, code formatting is crucial, with a focus on maintaining consistent indentation and spacing between lines of code. This not only enhances readability but also contributes to an overall neat and organized appearance. Commenting plays a significant role in our coding practices, as we ensure that comments are clear, concise, and directly relevant to the code they accompany. These comments serve as a valuable resource for other team members, allowing them to gain a thorough understanding of the code's purpose and how it has been implemented. By adhering to these coding guidelines, we strive to create code that is not only visually appealing but also facilitates effective collaboration and comprehension among team members.

Code Featuring QA and Usability Test

1.

Code under review

```
package com.example.springbackendv4.controller;

import com.example.springbackendv4.exception.UserNotFoundException;
import com.example.springbackendv4.model.Friends;
import com.example.springbackendv4.repository.FriendsRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@CrossOrigin("http://54.215.216.18:3000/")
public class FriendsController {
    @Autowired
    private FriendsRepository friendsRepository;


    //   ---------------------- BASIC POST, GET, DELETE MAPPING BY UNIQUE ID ---------------------


    @PostMapping("/friend")
    Friends friends(@RequestBody Friends friends){
        return friendsRepository.save(friends);
    }
```

```
    @GetMapping("/friends")
    List<Friends> getAllFriends(){
        return friendsRepository.findAll();
    }

    @GetMapping("/friends/{id}")
    Friends getFriendsById(@PathVariable Long id){
        return  friendsRepository.findById(id)
                .orElseThrow(()-> new UserNotFoundException(id));
    }

    @DeleteMapping("/friends/{id}")
    String deleteFriendsById(@PathVariable Long id){
        if(!friendsRepository.existsById(id)){
            throw new UserNotFoundException(id);
        }

        friendsRepository.deleteById(id);
        return "Deleted friends with id: " + id;
    }



    //   ---------------------- MODIFIED MAPPING (refer to FriendRepository for more info) -------------



    @DeleteMapping("/deleteFriends/{user_id}/{friend_id}")
    String deleteFriends(@PathVariable Long user_id, @PathVariable Long friend_id){
        friendsRepository.deleteFriends(user_id, friend_id);
        friendsRepository.deleteFriends(friend_id, user_id);
        return "Deleted friends : " + user_id + " and " + friend_id;
    }

}
```

Code submission to member of the same team
Review conducted by Omar of Team 4:

A team member implemented code for the "viewing posts" feature, referred to as p1. This work was carried out in a separate feature branch. After completing the code, the members pushed their changes to the feature branch. To incorporate these changes into the main codebase, a pull request was created, providing a description of the modifications made. This pull request was then reviewed by another team member. Following the review process, the changes were merged into the integration branch, also known as the master branch. This integration branch serves as the central repository for combining various feature branches and ensuring the stability and functionality of the overall codebase. By following this workflow, the team effectively

manages code changes, facilitates collaboration and quality control, and ensures that the latest updates are seamlessly integrated into the main codebase for further development and deployment.

The code under review here has proper indentation promoting readability of the code. The presence of comments help understand the functionalities in the code. The name of the file serves its purpose to describe the contents inside it.

Code submission to member of different team

Review conducted by Leo of Team 5:

```
1. Naming Convention  - naming convention is great and
straightforward. I can tell what the file is for and what the
functions do.
2. Files - as mentioned in the naming convention, the file looks
good, very clean.
3. Commenting and Layout - I don't see too much comments, but as a
programmer, I can easily understand the layout.
4. Limited Use of Global Variables and Functions - there's only one
variable and it's private, great!
5. Indentation - formatted nicely and clean, I can see what's
within the class and each functions.
6. Exception Handling - there is error handling in necessary
functions.
7. Statements - great!
```

Reason for choosing this code

This code was chosen because this code implements one of the unique features in the website. Having the snippet of the important file reviewed helps confirm the team if the project is progressing in the right direction.

2.

Code under review

```
package com.example.springbackendv4.controller;

import com.example.springbackendv4.service.StorageService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.ByteArrayResource;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.util.ArrayList;

/*
* This class will handle requests for file
* upload, download, and deletion from the
* front end. Restrictions should be implemented
* later, so that only signed-in users can
* interact with the aws s3 bucket(s).
*/

@RestController
@RequestMapping("/file")
public class StorageController {

    @Autowired
    private StorageService service;

//    @Autowired
//    private AmazonS3 s3Client;
//
//    @Value("${aws.s3.bucket}")
//    private String bucketName;

    /* This function will handle file upload requests */
    @PostMapping("/upload/{userId}")
    public ResponseEntity<String> uploadFile(@PathVariable Long userId, @RequestParam(value = "file") MultipartFile file) {
        return new ResponseEntity<>(service.uploadFile(userId,file), HttpStatus.OK);
    }
```

```
    /* This function will handle file download requests */
    @GetMapping("/download/{fileName}")
    public ResponseEntity<ByteArrayResource> downloadFile(@PathVariable String fileName) {

//        ----------

        byte[] data = service.downloadFile(fileName);
        ByteArrayResource resource = new ByteArrayResource(data);

        return ResponseEntity
                .ok()
                .contentLength(data.length)
                .header("Content-type", "application/octet-stream")
                .header("Content-disposition, ", "attachment; filename=\"" + fileName + "\"")
                .body(resource);
    }

    /* This function will handle file deletion requests */
    @DeleteMapping("/delete/{fileName}")
    public ResponseEntity<String> deleteFile(@PathVariable String fileName) {
        service.deleteFile(fileName);
        return new ResponseEntity<>("File deleted successfully", HttpStatus.OK);
    }

    @PostMapping("/restore/{fileName}")
    public ResponseEntity<String> restoreFile(@PathVariable String fileName) {
        service.restoreFile(fileName);
        return new ResponseEntity<>("File restored successfully", HttpStatus.OK);
    }

    @PostMapping("/moveFile/{source}/{destination}")
    public ResponseEntity<String> moveFile(@PathVariable String source, @PathVariable String destination) {
        service.moveFile(source,destination);
        return new ResponseEntity<>("File moved successfully", HttpStatus.OK);
    }

    @PostMapping("/moveFolder?source={source}&destination={destination}")
    public ResponseEntity<String> moveFolder(@RequestParam(value = "source") String source, @RequestParam(value = "destination") String destination){
        service.moveFolder(source,destination);
        return new ResponseEntity<>("Folder moved successfully", HttpStatus.OK);
    }

    @DeleteMapping("/trash")
    public ResponseEntity<String> emptyTrash(){
        service.emptyTrash();
        return new ResponseEntity<>("Trash emptied successfully", HttpStatus.OK);
    }
```

```
        /* This endpoint loads the user's saved file directory
        *  The object keys (bucket paths) are returned as an array*/
        @GetMapping("/getUserDir/{userName}")
        public ArrayList<String> listBucket(@PathVariable String userName) {
            return service.getRoot(userName);
        }

        @Scheduled(cron = "0 0 0 * * *")
        public ResponseEntity<String> automaticFileDeletion(){
            service.emptyTrash();
            return new ResponseEntity<>("Trash automatically emptied successfully", HttpStatus.OK);
        }


    }
```

Code submission to member of the same team

Review conducted by Josh of Team 4:

The naming convention for this file/code is consistent with our other files with camelcase for variables, methods, and classes. The literal names for functions are clear and

describe the functions' purpose making the code easy to read and understandable even if someone was to look at this code for the first time. Comments are provided above functions that clearly explain the functions purpose, helping developers who would need to edit the code understand the functionality without having to do extensive research. Layout of the code is appropriate as there is spacing between blocks of code such as functions, to provide readability. The code is also formatted correctly, follows proper indentation practices, well organized, and has imports at the top that are organized.Exception handling is present ensuring any exceptions are caught and handled appropriately. Statements are clear and each function has a specific task. Overall all of these practices contribute to the code's readability and maintainability.

Code submission to member of different team

Review conducted by Johnathan of Team 6:

1. Naming Convention:

The naming convention for the given code snippet is understandable and flows well. Variables are easy to read with camel case naming conventions.

2. Files:

Since their code was written in java, their file name for the code snippet shown is StorageController.java which is the proper java extension. Their main class within this file is StorageController which aligns with their file name which is a good convention to have.

3. Commenting and Layout:

The commenting within the code is very useful to understand what is going on through important functions within the file. Single line comments are also aligned with the given code methods too which is a nice layout.

4. Limited Use of Global Variables and Functions:
I see within the code snippet that there is a limited amount of global variables being used which is good. There are a lot of functions within this code snippet; however, I see the purpose that each function server.

5. Indentation:
Indentation within the code snippet looks nice and formatted, I know there are different methods of indentation; however, overall I can clearly specify where a function starts and ends, and what is being returned within the function–based on the proper indentation done.

6. Exception Handling:

Based on the code snippet, I do not see exception handling for cases which a user wants to upload, delete, or download–i.e no display of message in the case something goes wrong within these methods for the user. This is to be taken with a grain of salt as well because I see these features are only intended for signed-in users.

7. Statement:
Based on the code snippet, I see proper usage of classes and class variable declaration–private variables. I also see no usage of dangerous do-while struct loops which is good.

Signed:
Jonathan Valadez
Team Lead - Team 6

Reason for choosing this code
The reason for choosing the above code is because this file contains functionalities that work with storage. Since configuring a storage system is a significant part of our website, reviewing the above code would verify if it is along the right lines and provide ideas for steps to take moving forward.

Self-check on best practices for security

Protected Major Assets

Major Assets we are protecting include:

1. Account Information
    a. First and Last Name
    b. Password
    c. Intellectual Property of Student (uploaded resource)
2. AWS EC2 Instance
    a. SSH Username and Password
    b. SSH Secret Key
    c. SSH URL
3. RDS MySql
    a. Database Hostname
    b. Username and Password to connect to database

Confirmation of Encrypted Password

First, we have imported the BCryptPasswordEncoder framework, which is a PasswordEncoder that uses the BCrypt strong hashing function. We declare a new BCryptPasswordEncoder object with the strength of 16 (log rounds).

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class UserServiceImpl implements UserService {

    private final BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(16);
```

The encoder is then used in the same file in a function called encryptPasswordAndCreateUser, which will encrypt the password that is typed in by the user during sign in. The encrypted password is then tied to the user and later is stored into the database.

```java
@Override
public String encryptPasswordAndCreateUser(User newUser) throws UsernameAlreadyExistsException, EmailAlreadyExistsException {

    //Check if username is unique
    User userWithUsername = userRepository.findByUser(newUser.getUser());
    if(userWithUsername != null){
        throw new UsernameAlreadyExistsException(newUser.getUser());
    }

    //Check if email is unique
    User userWithEmail = userRepository.findByEmail(newUser.getEmail());
    if(userWithEmail != null){
        throw new EmailAlreadyExistsException(newUser.getEmail());
    }

    String newPassword = encoder.encode(newUser.getPassword());
    newUser.setPassword(newPassword);
    generateToken(newUser);
    User savedUser = userRepository.save(newUser);
    return savedUser.getId().toString();
}
```

Here's an example of passwords being stored in the database:

id	email	first_name	last_name	password	user	storage_left_in_by...
43	omar@yahoo.com	Omar	Thiongane	$2a$16$Wr.9O0yr...	omar	1000000000
42	wilson@yahoo.com	Wilson	Yin	$2a$16$4Z2YAsVz...	wilson	1000000000
41	joshDelRosario@yahoo.com	Joshua	DelRosario	$2a$16$f3Gnaahj....	josh	1000000000
40	saru@yahoo.com	Saru	Shrestha	$2a$16$eHegWnj5...	saru	1000000000
39	elias@yahoo.com	Elias	Abay	$2a$16$Q1m73FE...	eabay42	1000000000
38	kilian@yahoo.com	Kilian	Kistenbroker	$2a$16$Ap4c4J3e...	kilian42	1000000000
37	parthgeek@gmail.com	Parth	Panchal	$2a$16$.bYHt1y8u...	parth	1000000000

Confirmation of Input Data Validation

Input data that is being validated includes:

1. Username
    a. Regular expression is set to ensure the username will be 4 to 24 characters long.

```javascript
const user_regex_1 = /\S{4,24}/;
```

2. Password
    a. Three regular expressions are set to ensure:
        i. Password is 8 to 24 characters long
        ii. Password contains at least one letter
        iii. Password contains at least one number

```javascript
const password_regex_1 = /\S{8,24}/;
const password_regex_sl = /[a-zA-Z]/;
const password_regex_sn = /[0-9]/;
```

```
<div
  id="pwdnote"
  className={passwordFocus ? "account-instructions" : "offscreen"}
>
  <div className="message_spacing">
    <span className={password_regex_l.test(password) ? "valid" : "hide"}>
      <FontAwesomeIcon icon={faCheck} />
    </span>
    <span
      className={password_regex_l.test(password) ? "hide" : "invalid"}
    >
      <FontAwesomeIcon icon={faTimes} />
    </span>
    Password must contain 8 to 24 characters.
  </div>
  <div className="message_spacing">
    <span className={password_regex_sn.test(password) ? "valid" : "hide"}>
      <FontAwesomeIcon icon={faCheck} />
    </span>
    <span
      className={password_regex_sn.test(password) ? "hide" : "invalid"}
    >
      <FontAwesomeIcon icon={faTimes} />
    </span>
    Password must contain at least 1 number.
  </div>
  <div className="">
    <span className={password_regex_sl.test(password) ? "valid" : "hide"}>
      <FontAwesomeIcon icon={faCheck} />
    </span>
    <span
      className={password_regex_sl.test(password) ? "hide" : "invalid"}
    >
      <FontAwesomeIcon icon={faTimes} />
    </span>
    Password must contain at least 1 letter.
  </div>
</div>
```

3. Email
    a. A regular expression is set to ensure email is

```
const email_regex = /^\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*$/;
```

4. Search
    a. A regular expression for user search is set to ensure:
        i. Handle username search
        ii. Handle full name search

iii. Handle search with partial parameter value

```
10 usages  Elias Abay +1 *
public interface RegisteredUserRepository extends JpaRepository<RegisteredUser, Long> {
    1 usage  Elias Abay *
    @Query(value = "SELECT * FROM registered_user WHERE user LIKE :user OR email LIKE :user", nativeQuery = true)
    RegisteredUser findUsers(@Param("user") String user);
```

b. A regular expression for resource search is set to ensure:
   i. Handle folder/files' full name search
   ii. Handle search with partial parameter value

```
public interface UserResourceRepository extends JpaRepository<UserResource, Long> {

    @Query(value = "SELECT * FROM user_resource WHERE visibility LIKE 'Public' AND type LIKE 'Folder'", nativeQuery = true)
    List<UserResource> findAllPublicFiles();


    @Query(value = "SELECT * FROM user_resource WHERE REPLACE(filename, ' ', '') LIKE '%'  :query  '%' AND visibility LIKE 'Publ
    List<UserResource> searchByFilename(@Param("query") String query);
}
```

Self-check: Adherence to original Non-functional specs

S.N.	Non-functional specifications	Progress Report	Remarks
1	System shall always hide registered users account password information on startup the *Account* page.	Completed	Completed
2	System shall periodically delete files from a trash bin.	Completed	Completed
3	System shall provide information to registered users the amount of free storage remaining.	Completed	Completed
4	System shall display an error message when a registered user exceeds the cloud storage limit for their account.	Completed	Completed
5	Any response to StudentSources links shall be within 2 seconds.	Completed	Completed
6	Any response to any StudentSources links shall be within 4 seconds during high volume.	Completed	Did not have an environment to test during high volume.
7	The website shall be able to load text and image files with large sizes within 3 seconds and audio and video with large sizes within 10-15 seconds.	Completed	Completed
8	The website shall be able to load text and image files with large sizes within 5 seconds and audio and video with large sizes within 20 seconds during high volume.	Completed	Completed

9	Log-in and logout of websites shall be almost instant.	Completed	Completed
10	Buttons, textboxes, and options on the landing UI shall be responsive.	Completed	Completed
11	Buttons, textboxes, and options on upload content UI shall be responsive.	Completed	Completed
12	StudentSources shall be able to handle a large number of simultaneously active users.	Completed	Completed
13	StudentSources shall be able to handle files with a large size, with the max size being 1 GB.	Completed	Completed
14	The cloud storage container shall properly store files in TXT format.	Completed	Completed
15	The cloud storage container shall properly store files in PDF format.	Completed	Completed
16	The cloud storage container shall properly store files in MP3 format.	Completed	Completed
17	The cloud storage container shall properly store files in URL format.	Completed	Completed
18	The cloud storage container shall properly store files in JPEG format.	Completed	Completed
19	The cloud storage container shall properly store files in JPG format.	Completed	Completed
20	The cloud storage container shall properly store files in PNG format.	Completed	Completed
21	The cloud storage container shall properly store files in GIF format.	Completed	Completed

22	The cloud storage container shall be able to handle different file formats while still being efficient.	Completed	Completed
23	The database and cloud storage shall store metadata associated with files stored in the cloud storage container.	Completed	Completed
24	Users shall have a registered account to have access to all StudentSources files.	Completed	Completed
25	Passwords shall be encrypted in the database.	Completed	Completed
26	The website shall use secure hosting (AWS).	Completed	Completed
27	The website shall ensure only registered users can access resources.	Completed	Completed
28	The website shall ensure only admins can access sensitive information.	Completed	Completed
29	The website shall be optimized for energy efficiency, optimizing images, using efficient code.	Completed	Completed
30	StudentSources shall encourage students to use electronic files instead of printing and using materials to reduce paper waste and the environmental impact of printing.	Completed	Completed
31	StudentSources shall encourage responsible use by prompting users to delete unwanted files when max capacity of the account is reached	Completed	Completed

	to reduce storage and bandwidth of the website.		
32	StudentSources shall be marketed primarily to students.	Completed	Completed
33	The website shall be marketed as a better way for students to store and organize study resources.	Completed	Completed
34	The website will be marketed as a way to reduce paper waste.	Completed	Completed
35	StudentSources shall comply with copyright laws, to ensure that when the website stores files without infringing the copyrights of others.	Completed	Completed
36	StudentSources shall obtain permission to use copyrighted material according to fair use and copyright laws.	Completed	Completed
37	StudentSources shall have a privacy policy explaining how personal data is collected, used, and protected, which is important for things like student information.	Completed	Completed
38	StudentSources shall have security measures to prevent unauthorized access of data stored in the website's database.	Completed	Completed
39	StudentSources shall have terms and conditions that govern the use of the website by users.	Completed	Completed
40	Terms and Conditions shall explain rules about uploading and downloading files and acceptable use of the site.	Completed	Completed

41	Content present on the website shall be mostly if not all related to academic courses.	Completed	Completed
42	StudentSources' landing page UI shall be simple for users to understand.	Completed	Completed
43	The website shall be easy to navigate, with users only spending no more than 3-5 minutes browsing around.	Completed	Completed
44	StudentSources UI shall utilize non-distracting colors that complement the content.	Completed	Completed
45	StudentSources UI for uploading content shall be easy to understand and navigate.	Completed	Completed
46	The website shall have a clean and minimalist design,avoiding clutter and aiming for a sleek appearance.	Completed	Completed
47	The website shall have no issues running when experiencing high volume.	Completed	This might depend on the hosting service as well, AWS.
48	The website shall maximize user experience in storing and organizing study resources.	Completed	Completed
49	StudentSources shall only collect data that is necessary for the website's purpose.	Completed	Completed
50	The website shall be accessible using branded computers such as Windows, Apple, Asus.	Completed	Completed
51	The website shall be accessible on a range of devices like desktop	Completed	Completed

	computers, laptops, tablets, smartphones.		
52	Website shall be compatible with operating systems such as Windows and macOS.	Completed	Completed
53	Website shall be compatible with a wide range of web browsers, including but not limited to: Chrome, Firefox, Edge, Safari.	Completed	It was noticed that some functionalities such as uploading multiple files at once via drag and drop behave differently than expected in some browsers
54	The website shall support uploading of a range of file formats, like TXT, PDF, MP3, URL, JPEG, JPG, PNG, GIF.	Completed	Completed
55	StudentSources shall have a user-centered design, focused on ensuring the website is easy to use and navigate for users.	Completed	Completed
56	The website shall have a clear purpose for students and stakeholders and not be too broad.	Completed	Completed
57	The source code shall be clear.	Completed	Completed
58	Source code shall be concise.	Completed	Completed
59	Source code shall be consistent.	Completed	Completed
60	Source code shall be well commented.	Completed	Completed
61	Source code file names shall be clear.	Completed	Completed
62	Source code file names shall be concise.	Completed	Completed

63	Source code file names shall be consistent.	Completed	Completed
64	The file structure of source code shall be easy to navigate.	Completed	Completed
65	Uploaded audio and linked audio and video files shall be streamed to promote better user experience.	Completed	Completed
67	System shall show a registered user a progress and estimated wait time upon uploading files.	Completed	Completed

Team Member Contribution

Milestone 1 - Contribution Detail

Team Member	**Contribution**
Saru Shrestha	- Use case diagram - Research competing products - Research SW Stack architecture and tools - Configure Host server - Configure Database server - Revise Milestone 1 documentation
Kilian Kistenbroker	- List functional requirement - Research SW Stack architecture and tools - Research important features of competitive products - Configure Host server - Backend (request/response) program
Elias Abay	- List non-functional requirement - Research key features of competitors vs. StudentSource
Parth Panchal	- Main use cases - Research key features of competitors vs. StudentSource - Manage official GitHub Repository - Set up and manage backup GitHub Repository
Joshua Del Rosario	- Main use cases - List functional requirement - List of main data items and entities - Ordering of non-functional requirement - Competitive Analysis Summary - Format Document
Wilson Yin	- List non-functional requirement - List functional requirement - List of main data items and entities
Omar Thiongane	- Main use cases - Use case diagram

	- Research important features of competitive products - Format Document

Milestone 2 - Contribution Detail

Team Member	**Contribution**
Saru Shrestha	- Data Definitions - Prioritized Functional Requirements - High level database architecture and organization (documentation) - Deployment Diagram - Identify Key risks - Project Management (documentation) - Design logo (outsourced) - Draft Terms and Conditions document - Revise Milestone 2 documentation
Kilian Kistenbroker	- UI Mockups and Storyboards - High Level UML Diagrams - Frontend design and implementation - Vertical Prototype
Elias Abay	- Data Definitions - High level database architecture and organization: Search/filter architecture (documentation) - Vertical Prototype
Parth Panchal	- UI Mockups and Storyboards - High Level UML Diagrams - Vertical Prototype - Maintain Back-up GitHub Repository
Joshua Del Rosario	- High level database architecture and organization (documentation) - Database implementation in MySQL database
Wilson Yin	- High level database architecture and organization (documentation) - High Level APIs and Main Algorithms (documentation) - High Level UML Diagrams
Omar Thiongane	- Application Networks Diagram - Deployment Diagram

Milestone 3 - Contribution Detail

Team Member	**Contribution**
Saru Shrestha	- Documentation - Wireframe - Update ERD - EER diagram - Update UML - Update Networks and Deployment Diagram - Gather image contents - Reference page (frontend) - Revise M3 documentation
Kilian Kistenbroker	- Plan and assign tasks for frontend - Render directory tree in left panel - Render current directory onto main panel - Implement functionality of ‘Create Folder’ - Implement functionality for uploading through drag-and-drop and upload button - Render file viewer of different file format - Implement functionality for downloading files from file viewer into users local downloads folder - Implement functionality for ‘pinning’ folders and files - Implement functionality for ‘create file’ for txt and url files - Implement search functionality for folders/files in a current directory - ‘Info’ tab that displays basic information of current folder/file - Render other users homepage (using dummy data) - General purpose pop-up windows for displaying warnings or errors that may occur - Review and debug code - Update names in database to be consistent with documentation - Integrate frontend, backend and

	database
Elias Abay	- Plan and assign backend task - Research file system - Configure file system - Generate root folder for newly registered users - Implement functionality to delete items from root folders - Review code - Integrate frontend, backend and database
Parth Panchal	- ‘Notes’ tab (frontend) - ‘Comments’ tab (frontend) - Implement functionality for ‘notes’ tab (backend) - Implement functionality for ‘comments’ tab (backend) - Maintain backup GitHub repository
Joshua Del Rosario	- Research file system - Configure file system - Implement functionality to upload and download files and folders in the file system - Implement functionality to return root folder of other registered users - Update names in database to be consistent with documentation - Revise ERD diagram
Wilson Yin	- Research file system - Configure file system - Implement functionality to retrieve files and folders from file system to the user end
Omar Thiongane	- ‘Notifications’ Panel (frontend) - ‘User Account Panel’ with features of logging out, displaying remaining storage, and displaying user info (frontend) - ‘Visibility and permissions’ Panel - Take notes during in-person review of prototype

Milestone 4 - Contribution Detail

Team Member	**Contribution**
Saru Shrestha	- updated functional and non-functional requirements in all milestone documents - kept track of committed functions and non-functional requirements - reviewed the rough draft of M4 documentation and drafted final draft of the document - recruited sample user for usability test - reached out to members of other teams to review code - reviewed code for Team 4, Team 5 and Team 6 - Revise Milestone 4 documentation
Kilian Kistenbroker	- (implementation of the following) - display user information in the Account panel. (frontend) - provide an option to update/delete information on the Account panel. (frontend) - create a splash message in the bottom right corner for updating, deleting, logging out, and logging in. (frontend) - provide indication of storage remaining on the user account panel. (frontend) - block upload request that exceeds the limit. (frontend) - implement a way to drag folders/files into other folders in the main viewer and directory tree.(frontend) - provide download, rename, and delete options.(frontend/backend) - render items in the trash bin onto the main viewer. (frontend) - configure visibility and permissions. (frontend)

	- Synced comments section and made comments unique per folder and file. (backend) - Synced notes and made notes section unique per folder/file. (backend) - Provided a system for direct/group messaging (frontend/backend) - Made public folders/files searchable. (frontend) - Provided download options when visiting pages where permissions are granted. (frontend) - implemented video download streaming with buffer range. (backend) - Created handling for moving folders/files between directories. (frontend) - Created JSON controller in for managing/manipulating user home directories (backend) - Provided loading progress indicator while files are uploading. (frontend) - sync frontend and backend - recruited sample user for usability test
Elias Abay	- (Implementation of the following - backend) - moving files and folders - generating links for sharing - periodically delete files from a trash bin - recruited sample user for usability test
Parth Panchal	- (Implementation of the following - backend) - handling storage limit - recover files from trash - track progress and estimated wait time after the file is uploaded - typing and saving notes - updating commenting feature to generate unique id and track it in

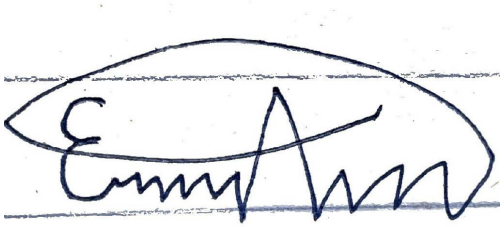
	database - password encryption when storing in database - recruited sample user for usability test - maintained backup repository
Joshua Del Rosario	- section 1, 5 and 6 of M4 documentation - reviewed code snippet 2
Wilson Yin	- (Implementation of the following - backend) - generate new file or folder after receiving request from frontend to backend - renaming files and folders - downloading directories - viewing shared directories
Omar Thiongane	- section 2 of M4 documentation - reviewed code snippet 1 - recruited sample user for usability test

Team Member Contribution Score

Team Member	Score
Kilian Kistenbroker	9.5
Elias Abay	8.5
Parth Panchal	9
Joshua Del Rosario	9
Wilson Yin	9
Omar Thiongane	9

Scale: 1 to 10, where a score of 1 means “no work at all”, and score of 10 means “contributed to the maximum capacity”

Team Members' Signature In Agreement of Contribution Score

Team Member	Signature
Kilian Kistenbroker	
Elias Abay	
Parth Panchal	
Joshua Del Rosario	
Wilson Yin	wy
Omar Thiongane	

Post Analysis - Lesson Learned

Main Challenges

This project was a fantastic experience for students like us trying to break into the Tech industry. As Computer Science students, we learned many new aspects we could incorporate into our Resume. As great as it was, this project was intensive for students learning the dos and hows of the real-world work environment. There were some challenges that the team faced during the duration of this project. The chief hindrance was time. A massive amount of functions were to be implemented in a short period. In addition to that, the members were new to the technologies being used. Hence, the research and learning of those technologies had to be done quickly, which also hurt the chances of understanding those technologies meticulously. There was a scenario where the team had to change technology for the file system and start over after investing half of the milestone's time in the previous technology. The reason for changing the technology was that after choosing the file system service, members discovered that there was difficulty finding reliable resources and references to learn about the service; hence, the team had to choose a new file system service and re-implement functions according to the new technology. Another issue that arose because of a shortage of time was not finding time for an in-depth code review. When the frontend and backend codes were merged, there was little to no time to review and debug codes rigorously. Apart from those, the members whose language skills in the chosen programming language were not as strong as other members could not get help as much as needed because every member had to use their time in researching the new technologies being used and balance their workload among other classes as well.
Apart from the obstacles that emerged due to time, there was another challenge for a team lead, checking the validity of the code. For a student team lead, there was limited knowledge about confirming source codes, like if the code was copyright-free or if the code was generated using AI chatbots.

Addressing Those Challenges

In regards to implementing numerous functions in a short time, compromising some functions might have made a difference. In terms of learning new technologies in a short time, it would have been better for Computer Science students to have a habit of adapting and playing around with new technologies often in their own time as well; therefore, they would have prior knowledge and valuable insights when initiating projects like this where the time is limited. When new technology is chosen, researching if there are abundant references related to the technology before committing to the technology might have lessened the negative impact while working with new

technology. To cope with another challenge of not finding enough time to review and debug the code, compromising functions and allocating a reasonable amount of time to review the code might have been a good approach. When it comes to helping students with lower programming language skills, more reference resources could have been provided, and the student could have been encouraged to dedicate their time to learning the language. Next, regarding filtering codes with copyrights or AI-generated, a document was created where members would list the resources from which they referenced their codes. However, if the source was not listed, then there was no other resource to confirm the validity and reliability of those codes. Reaching out to experienced experts like a professor might have helped in that situation.